

Intro to SLAM

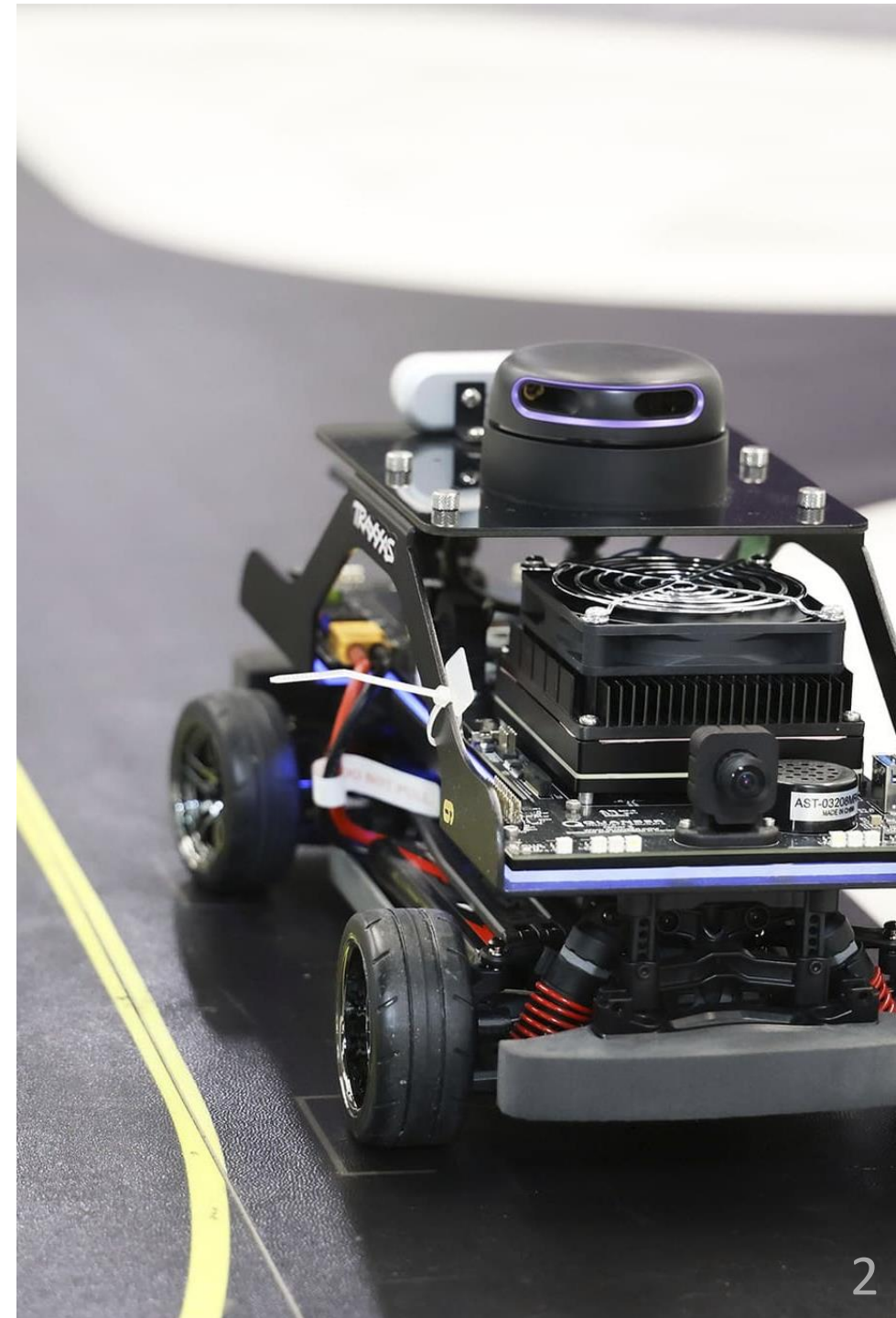
Technical project part II : Autonomous car

February 2026

Referent Professor
Dr Naila Bouchemal naila.bouchemal@ece.fr

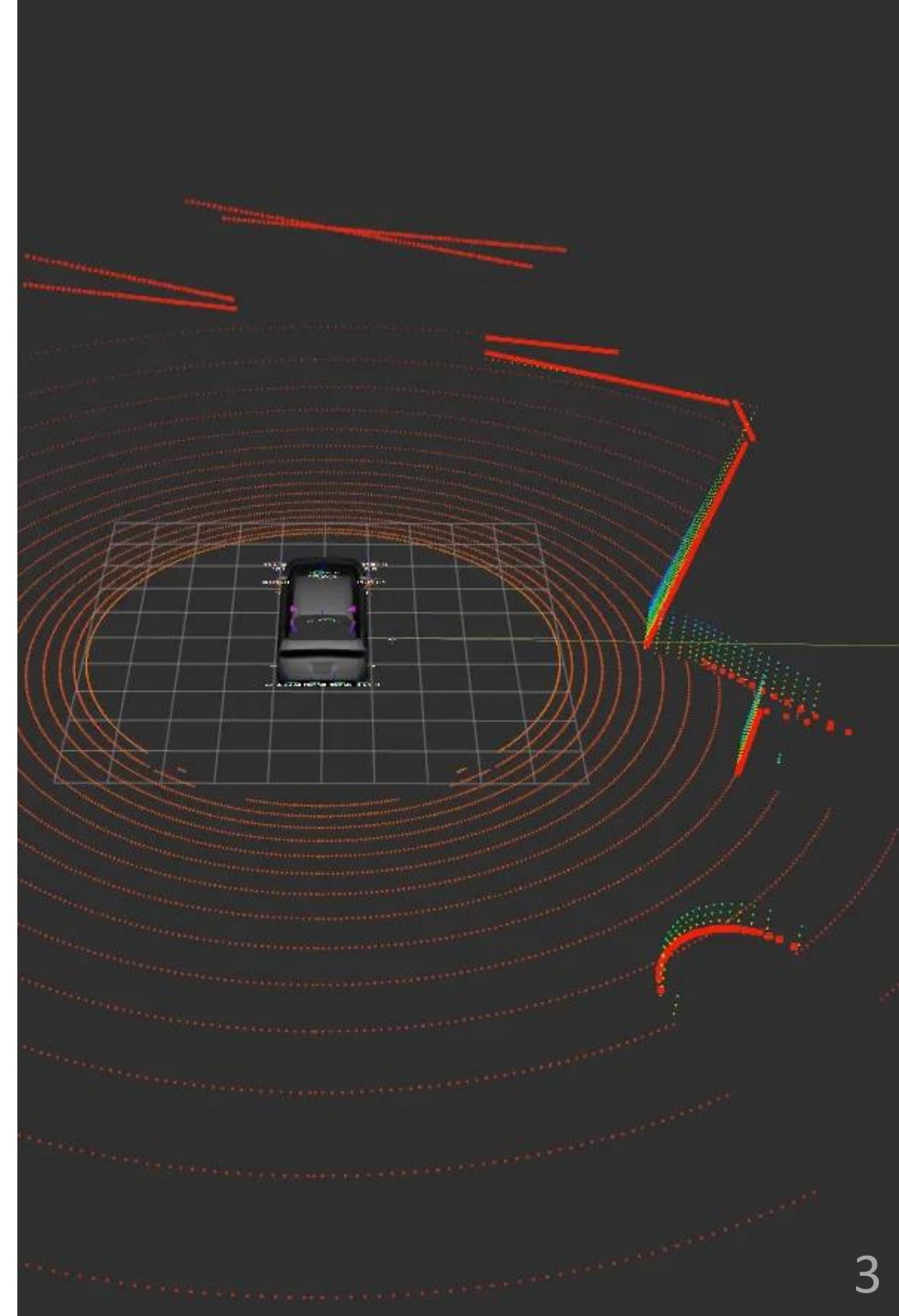
Goals of the lecture

1. Understand **what SLAM algorithms are**
2. Put some logics (and a tiny bit of Maths 😊) behind **Localization** and **Mapping techniques**
3. Guide you toward the use of **built-in libraries** for **your projects**



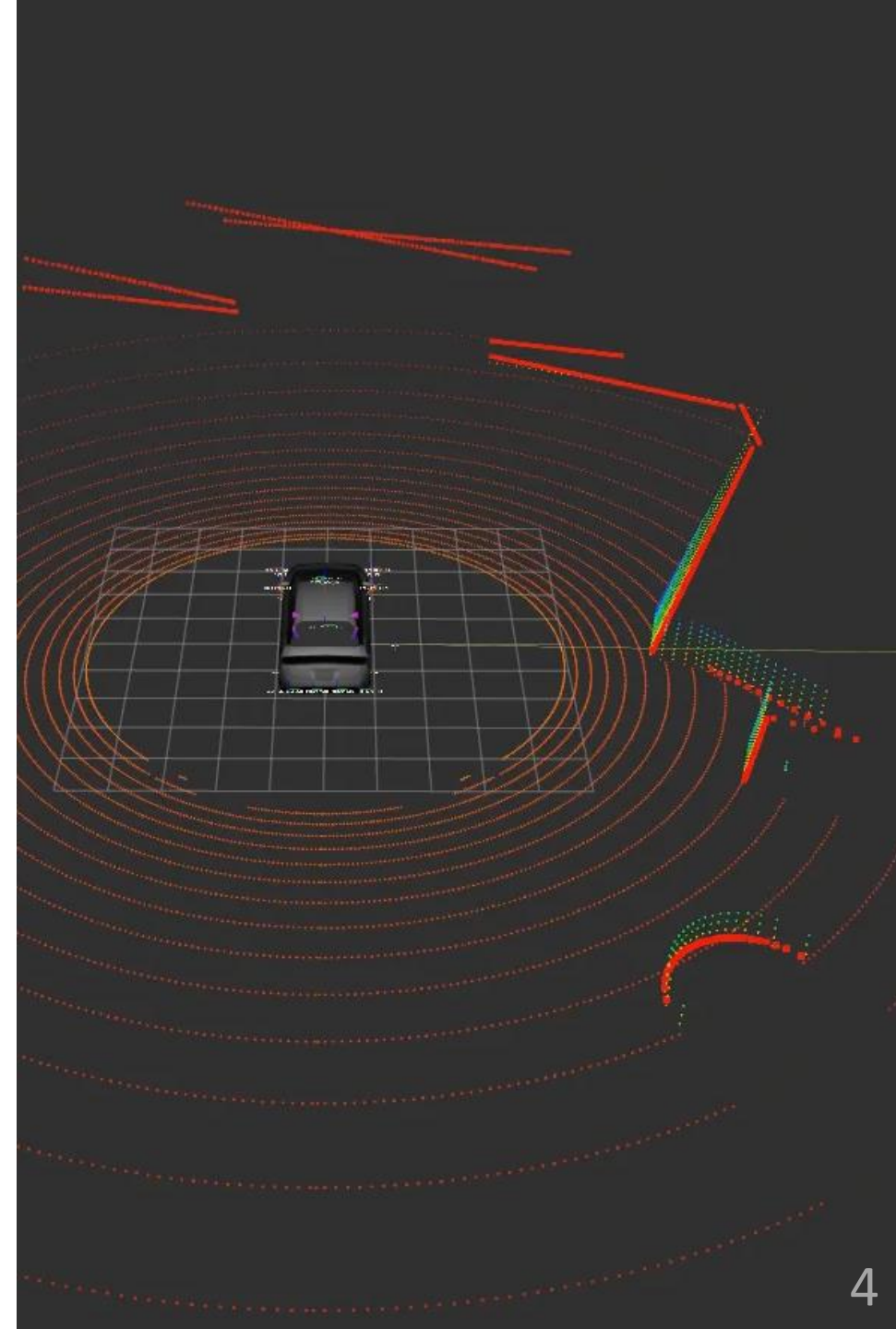
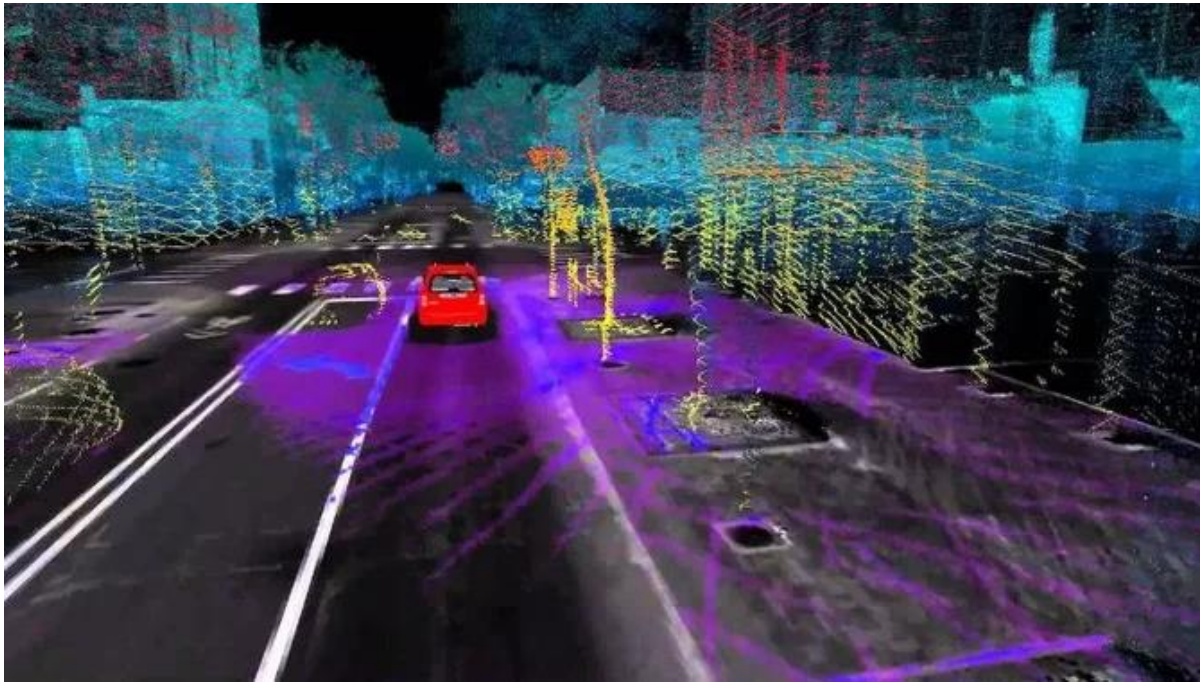
Plan

1. What is SLAM
2. The localization problem
3. The mapping problem
4. Doing altogether
5. Planning while mapping
6. Nav2 ROS2 library



(1/6) What is SLAM ?

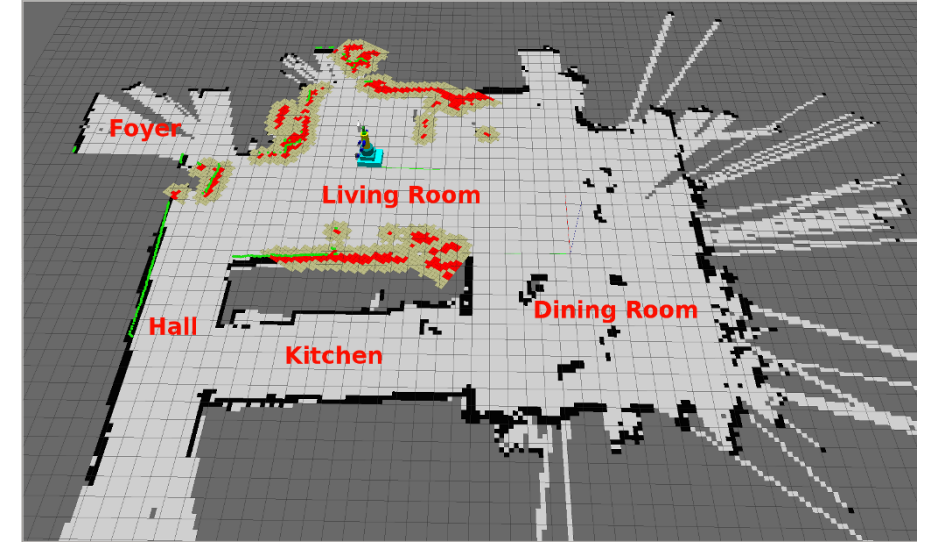
Simultaneous Localization And Mapping



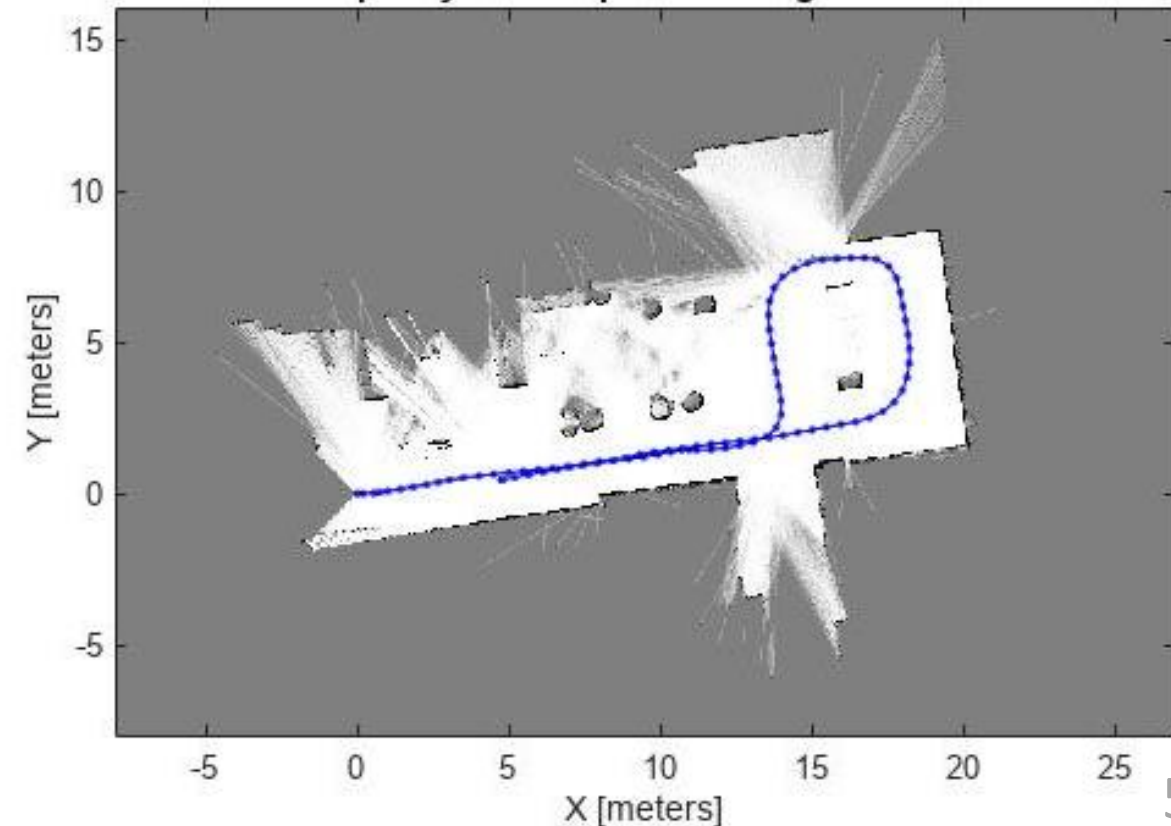
(1/6) What is the problem answered by SLAM ?

Simultaneous Localization And Mapping

- Robot is placed in an **unknown** environment
- Robot creates a **live map**
- Robot **locates itself** in the map
- Robot can **plan trajectories** within this map

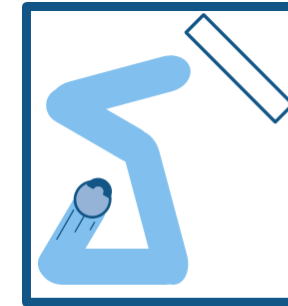
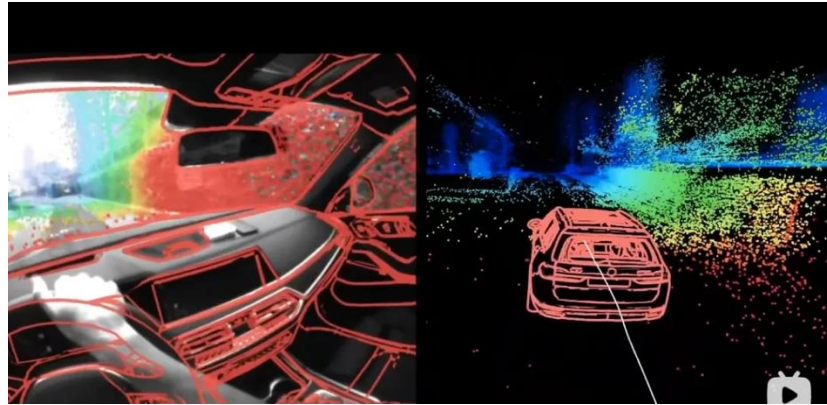


Occupancy Grid Map Built Using Lidar SLAM

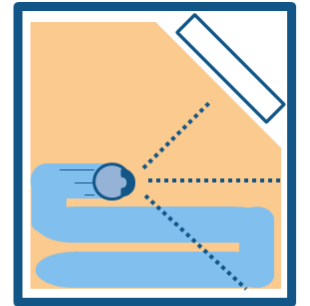


(1/6) Use of SLAM :

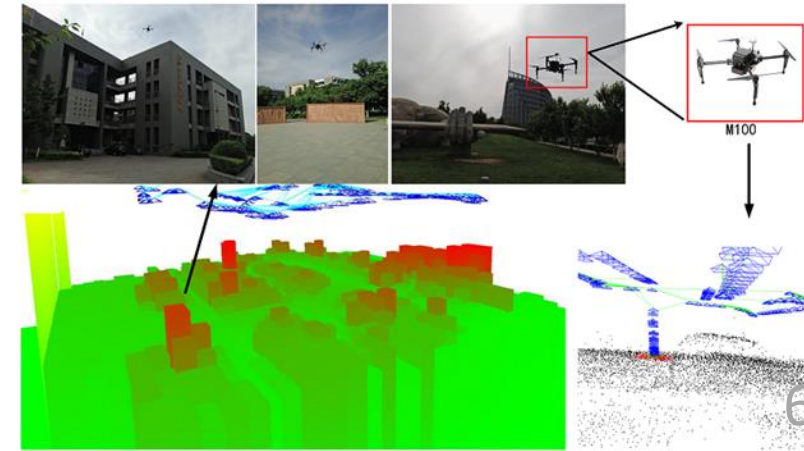
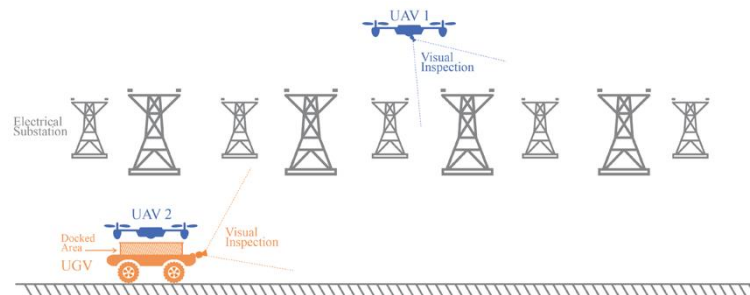
- Autonomous Driving
- Area coverage (rescue, defense, ...)
- Vacuum cleaner robots
- GPS-denied environment
- Mapping of unknown and constrained environments
- Multi-agents Collaboration framework : Plan + Acting



Without SLAM:
Cleaning a room randomly.

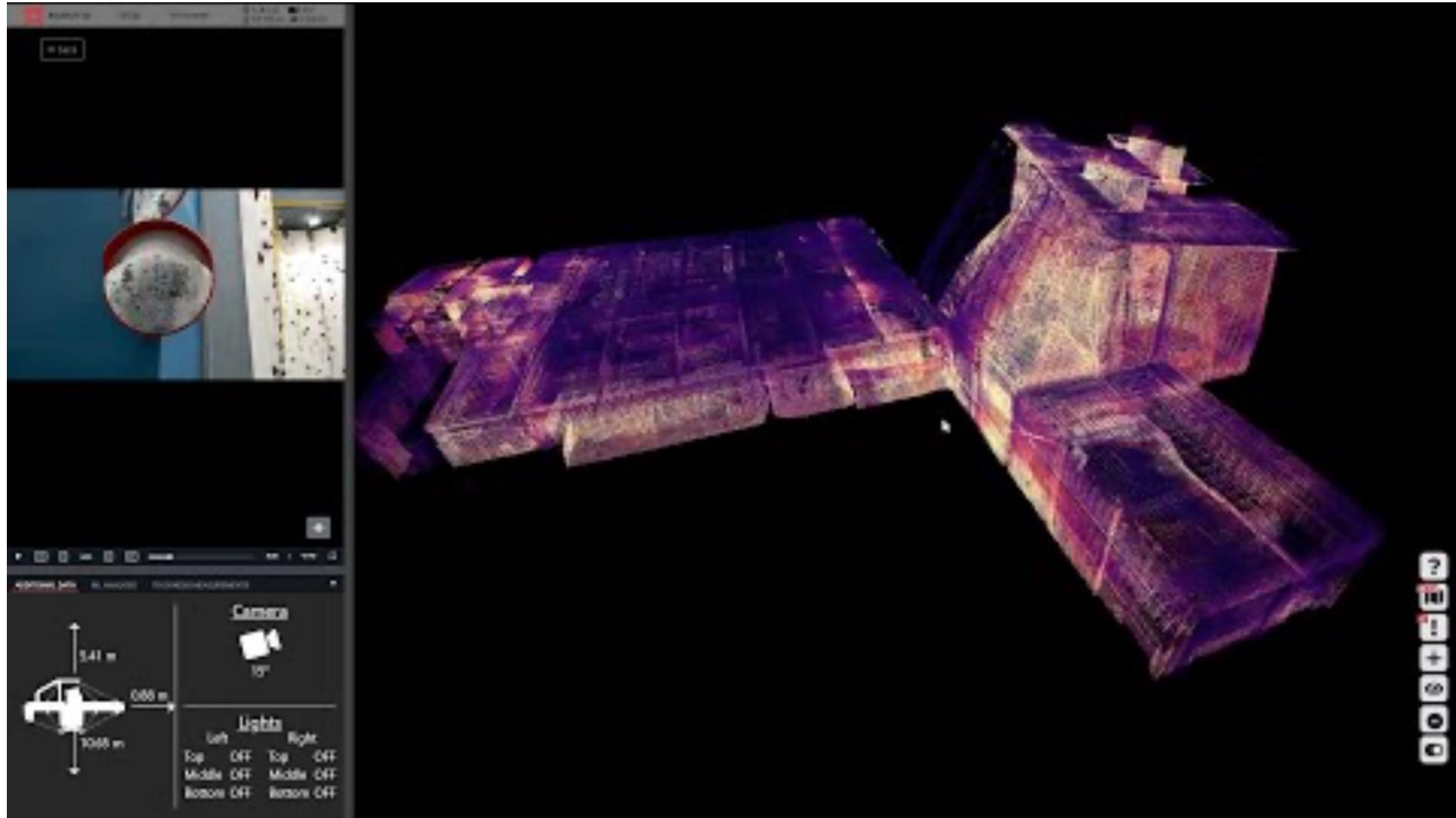


With SLAM:
Cleaning while understanding the room's layout.



(1/6) What is the problem answered by SLAM ?

SLAM example on a drone, localization is needed to place points correctly :

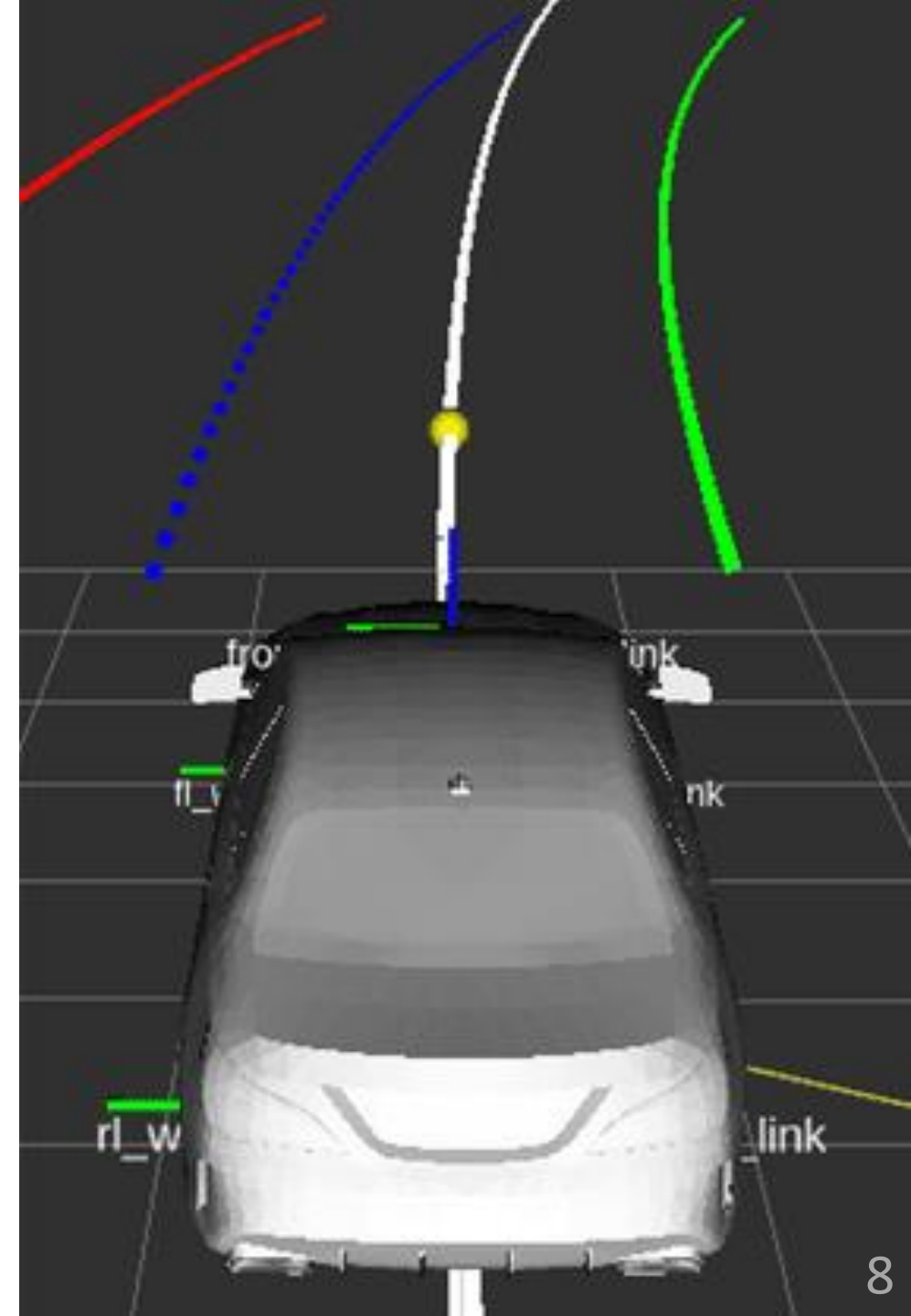


(1/6) What is SLAM ?

Simultaneous **Localization** And **Mapping**

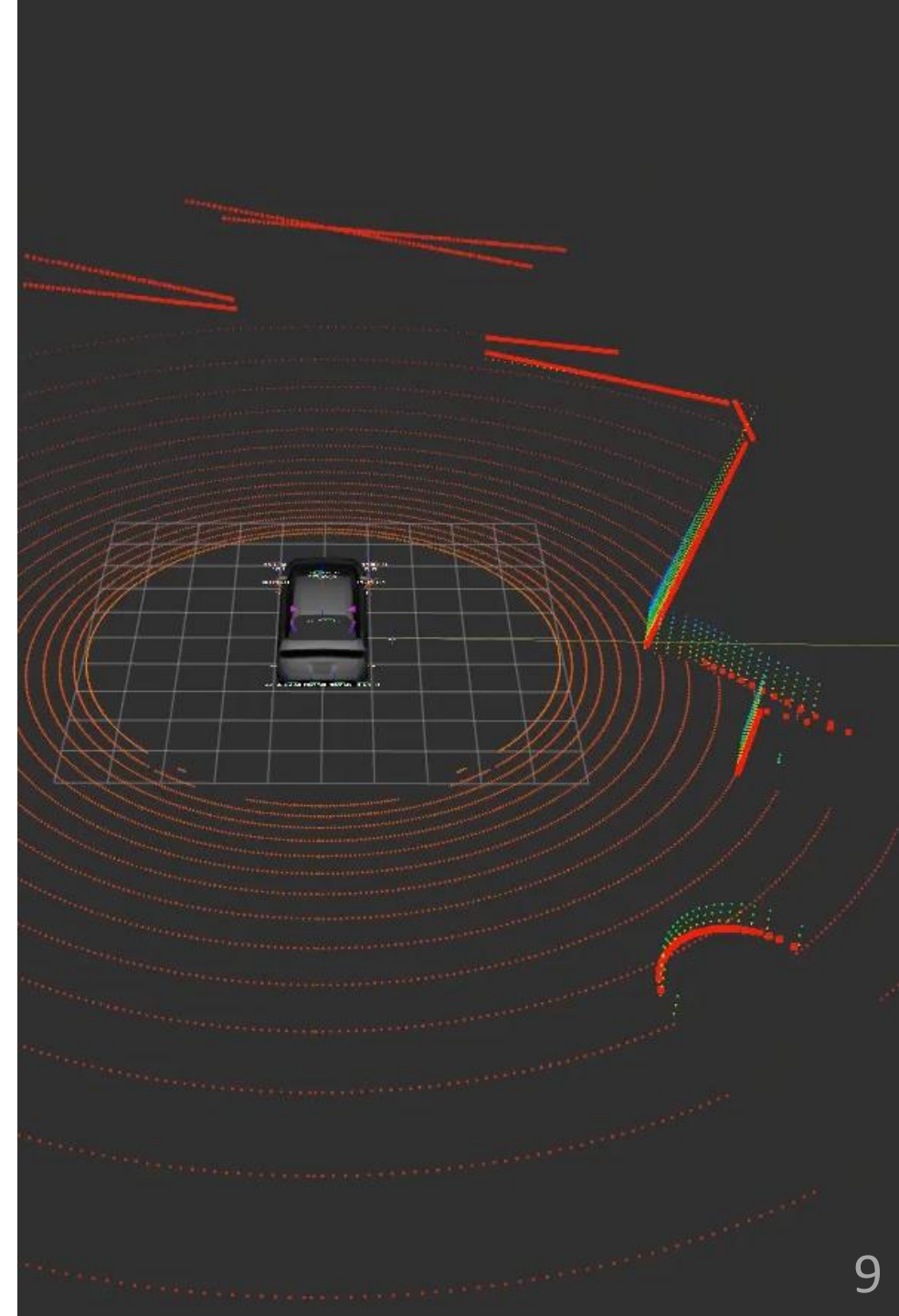
- Localization :
Absolute positioning of the robot in cartesian frame
 (x, y, z, θ)
- Mapping :
Generating a map (which type?) of the surrounding environment
 $\{points_i = (x_i, y_i, z_i)\}_{i=1 \text{ to } N}$
- Simultaneous :
Starting with no assumptions, localizing the drone on the current state of the map

Localization $\Leftrightarrow$ Mapping



Plan

1. What is SLAM
- 2. The localization problem**
3. The mapping problem
4. Doing altogether
5. Planning while mapping
6. Nav2 ROS2 library



(2/6) The Localization problem

Localization

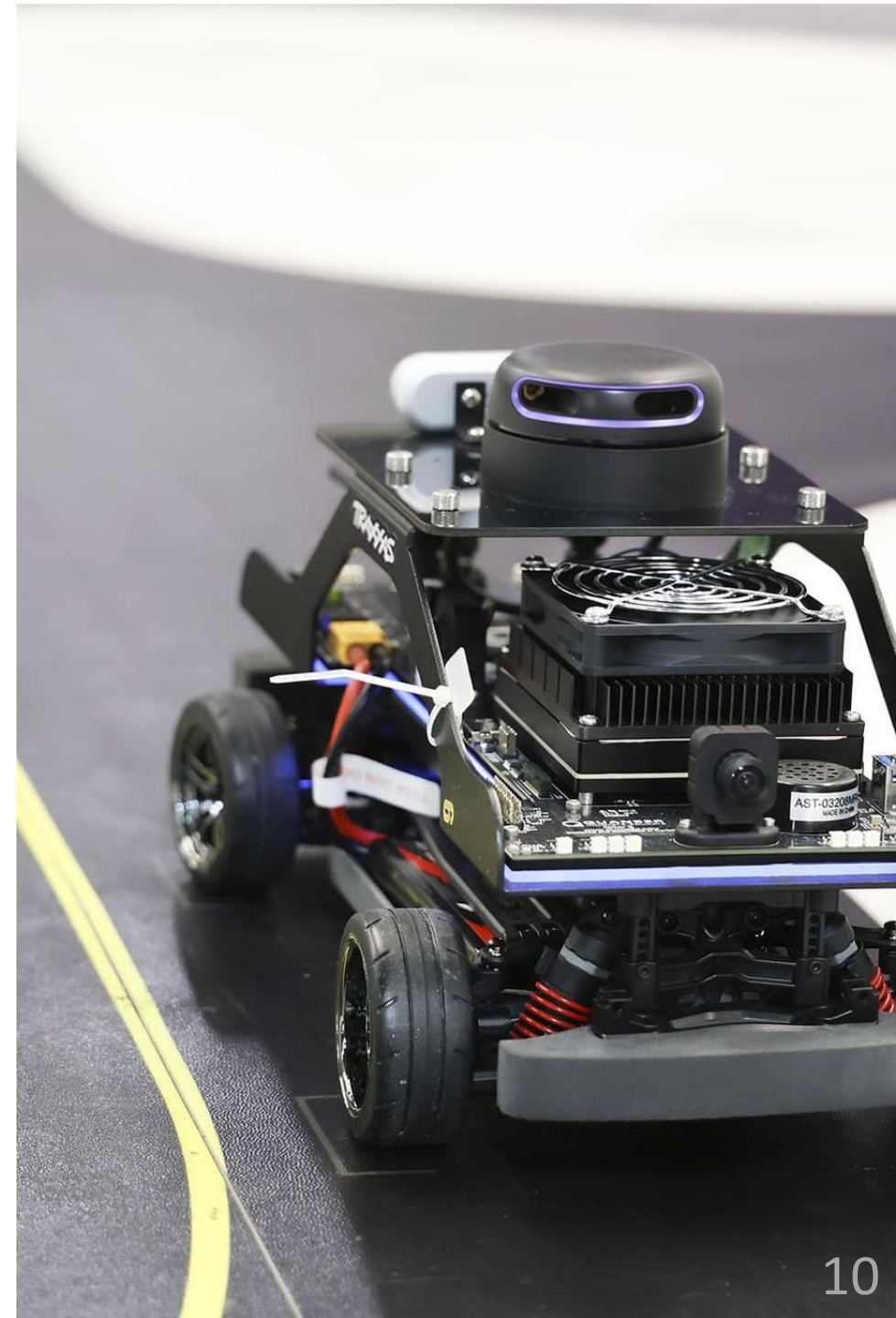
Estimating an object's pose (which is ?) relative to a map of its environment.

Some problems :

- Position tracking (position or position/attitude) from a known starting point
- Global localization from scratch
- Kidnapping

Can be in a **static** or **dynamic** environment

Via **passive** or **active** strategies

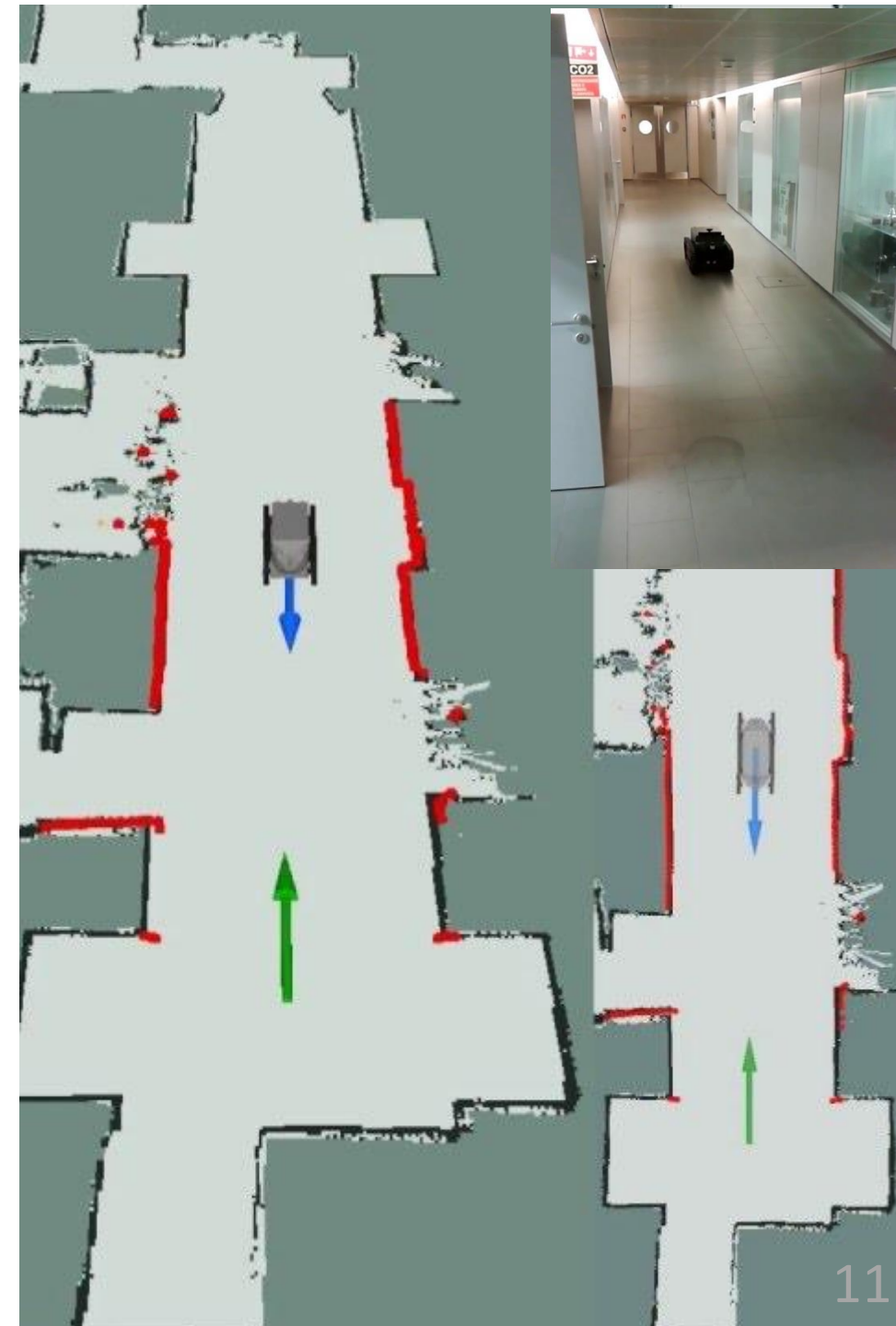
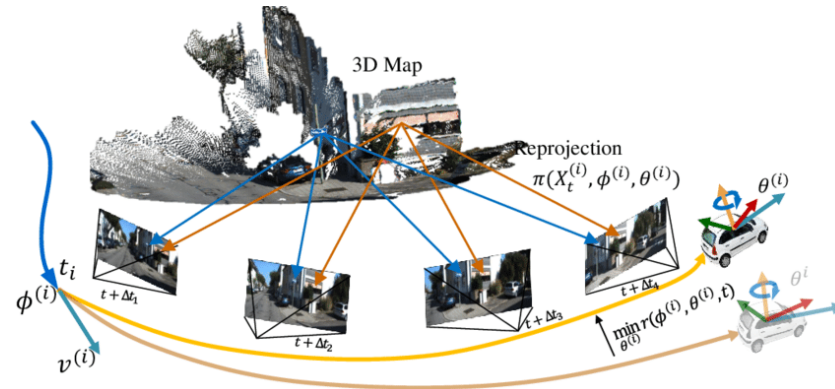


(2/6) The Localization problem

Localization

Sensors

- GPS (Absolute position + noise)
- Beacon positioning / triangulation (Absolute position)
- RGB/Depth vision : Visual Odometry (Position or speed)
- Optical flow (Speed)
- LiDAR : Visual Odometry
- IMU (Acceleration)
- Gyroscope (Angle rates)
- Wheels/Motor encoders : Odometry
- Visual odometry ->
- ...

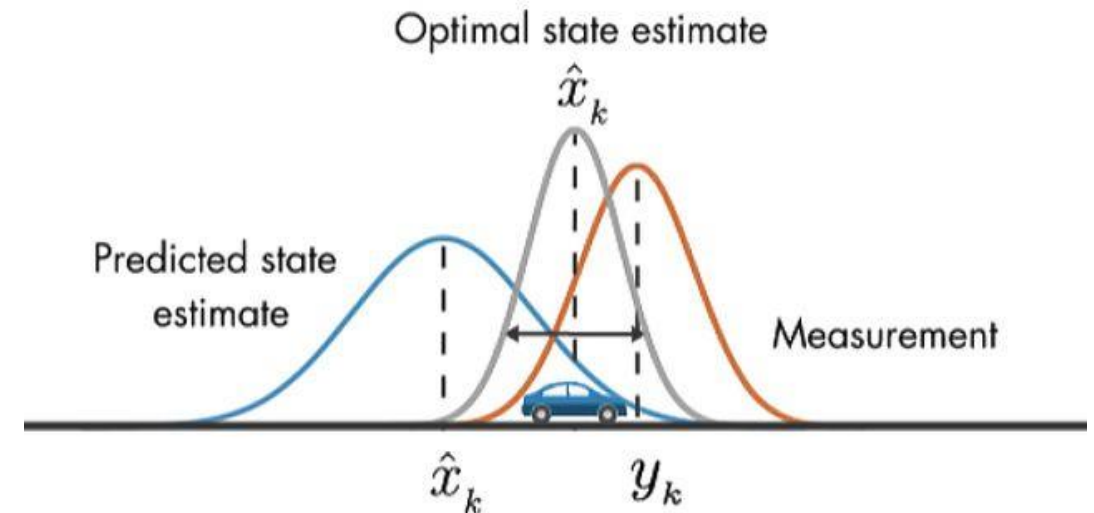
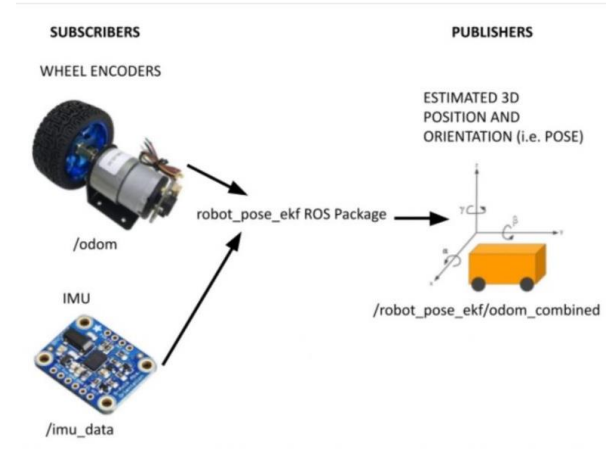
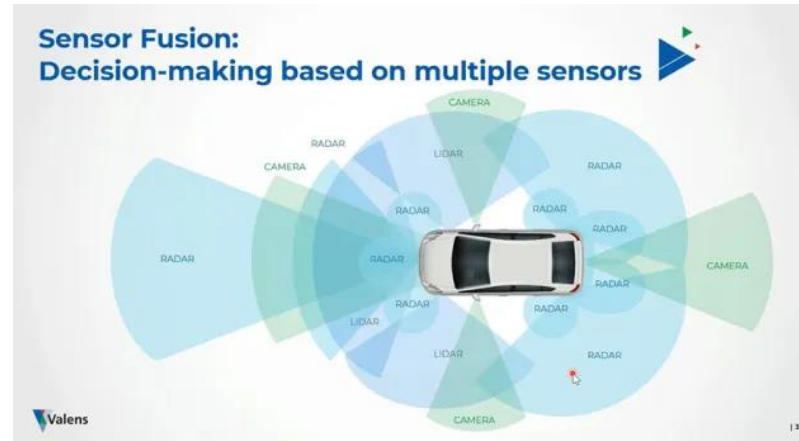


(2/6) The Localization problem

Localization

The main idea : Fusing

1. We use all available sensors
 - Different frequencies
 - Different information
 - Different noises
2. We leverage our model of the world
 - Dynamics model
 - Maps
 - ...
3. We **fuse** everything, we “average” results



(2/6) The Localization problem

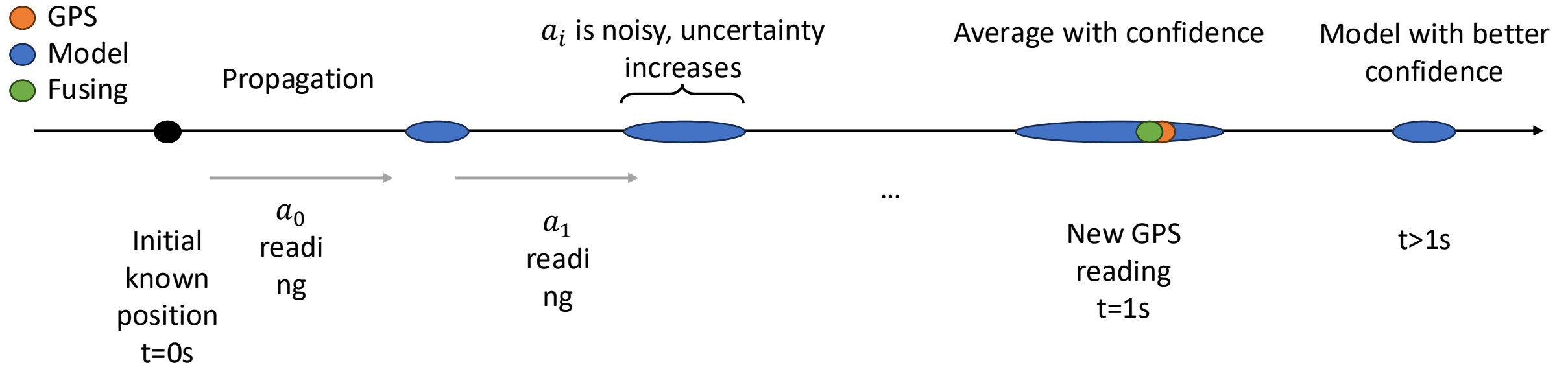
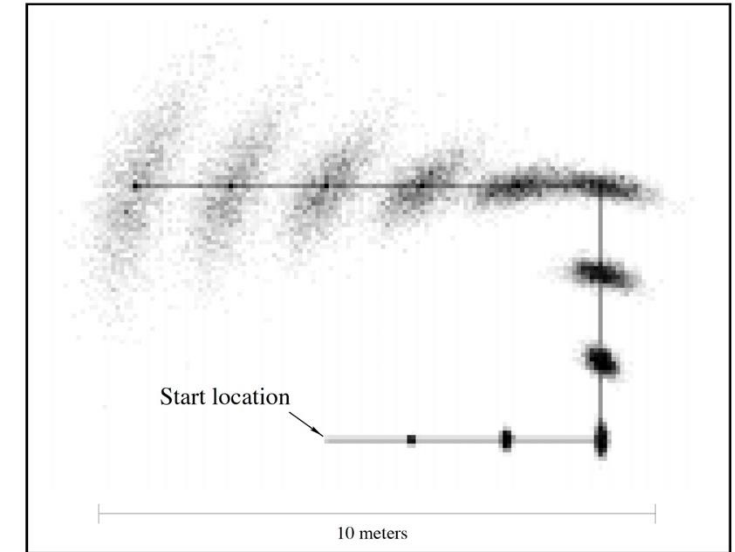
Localization – Kalman Filtering

The main idea : Fusing

On a 1D car :

- GPS @ 1 Hz : Position, very low noise
- IMU @ 100Hz : Acceleration, very noisy ($\sigma_{IMU} \gg \sigma_{GPS}$)
- Model : $x_{k+1} = x_k + v_k \Delta t + a_k \frac{1}{2} \Delta t^2$, $v_{k+1} = v_k + a_k \Delta t$

2D odometry with no GPS:

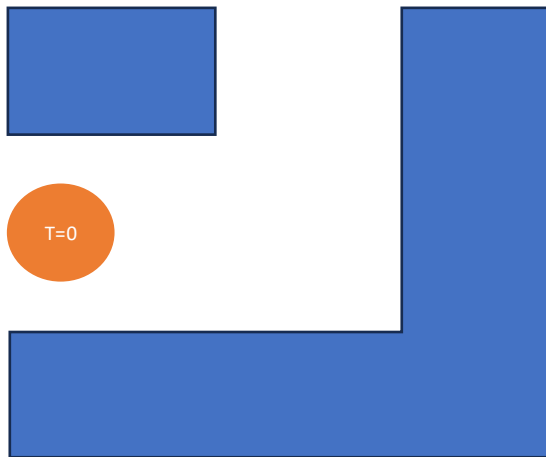


We can estimate both the state AND its uncertainty !

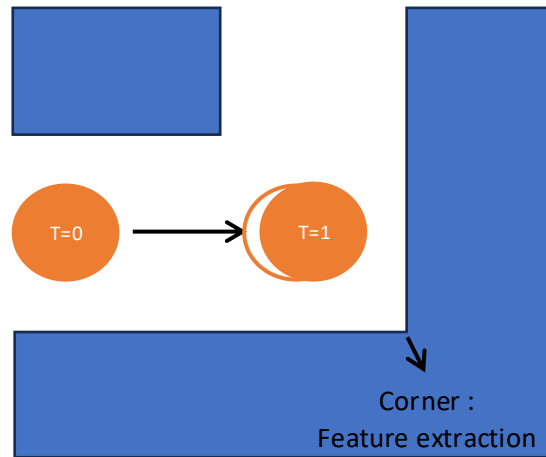
(2/6) The Localization problem

Note From relative to absolute localization

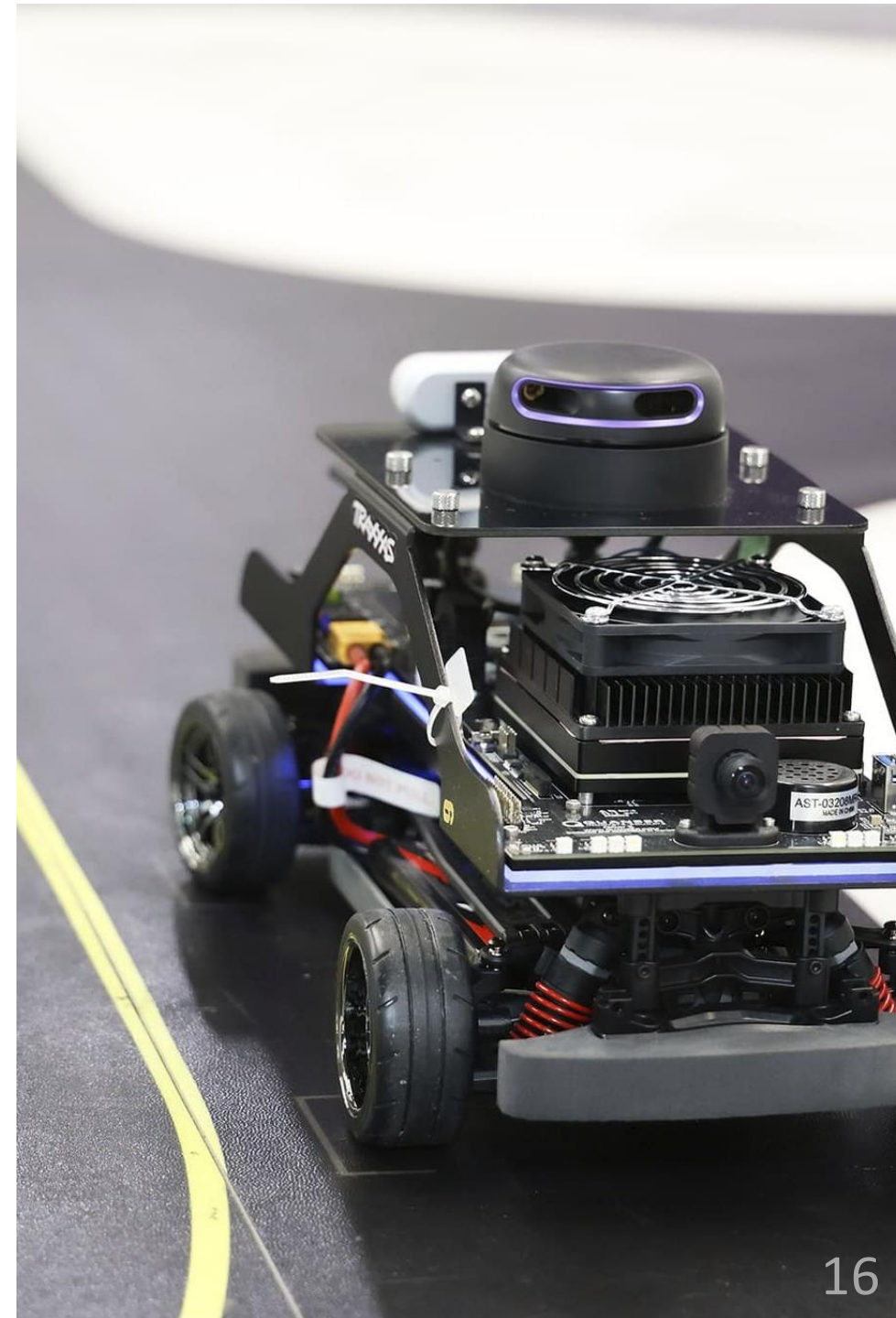
If we have access to a map, it can be integrated in the sensor fusion:



Absolution position
(0 ,0)

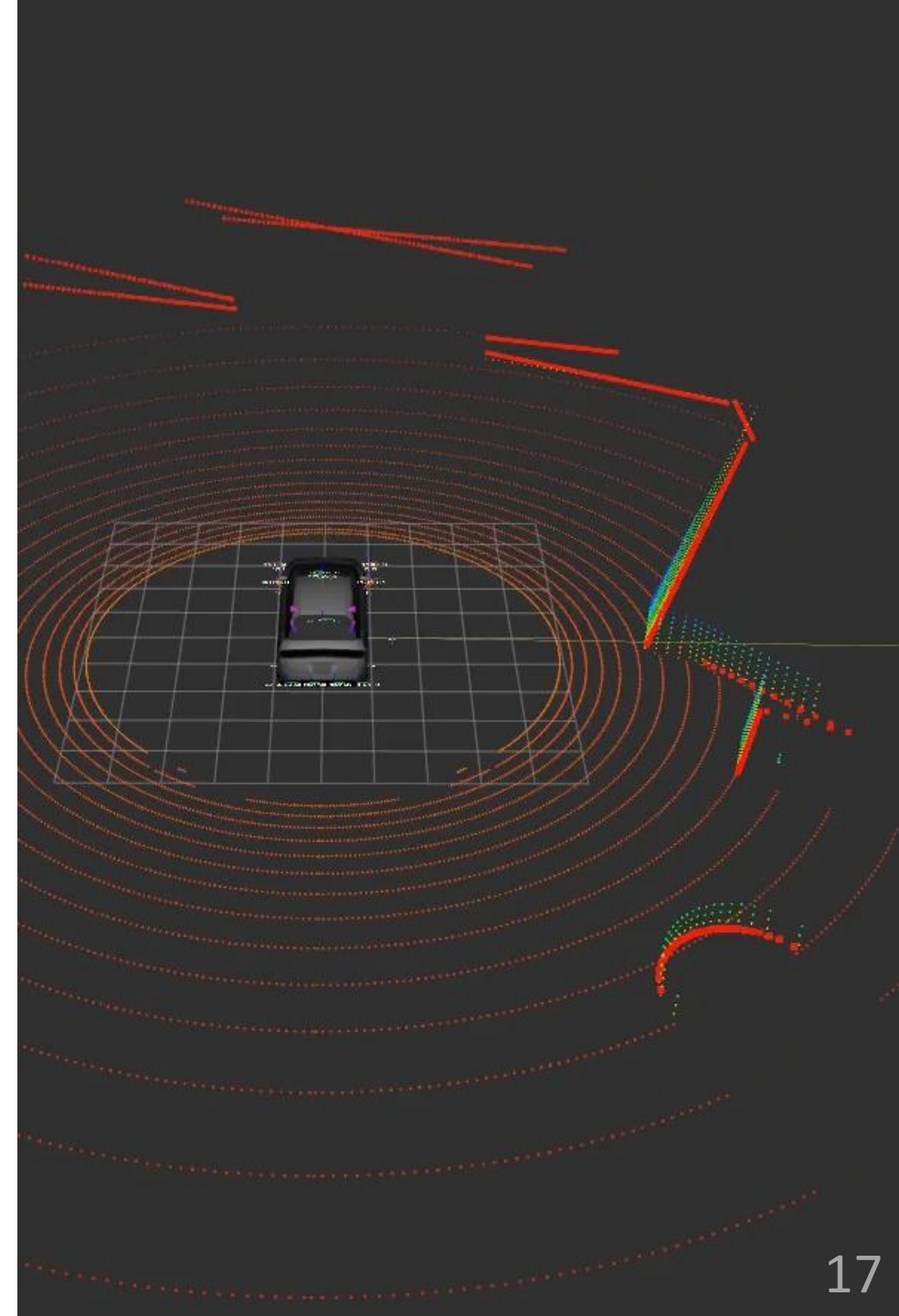


Relative movement fromIMU:
(+ 0.9, 0)
Map can be used to infer the
absolute position
(1.0,0)



Plan

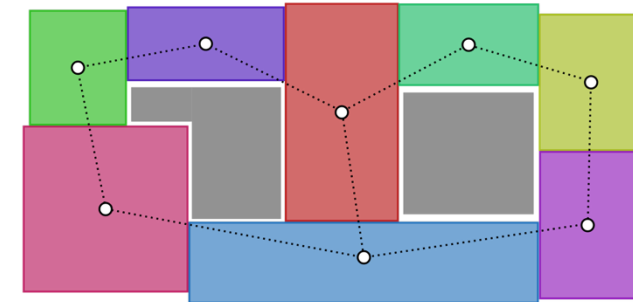
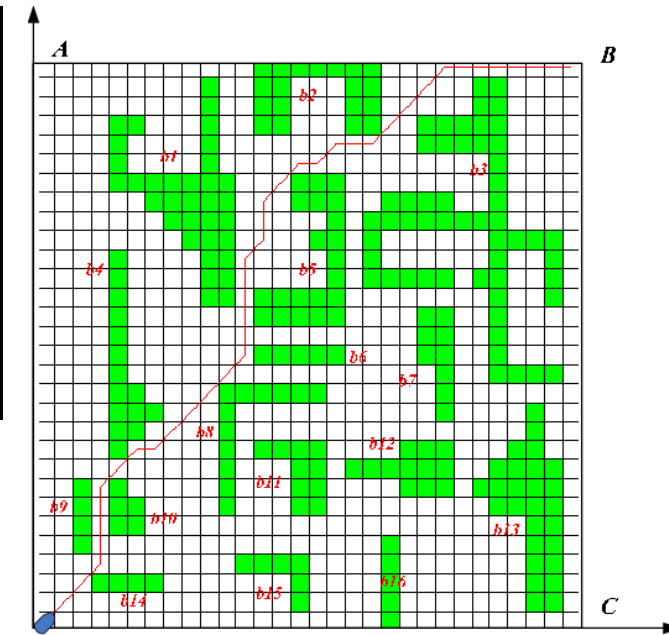
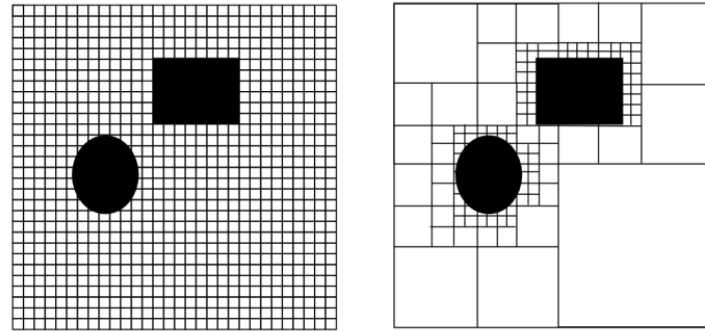
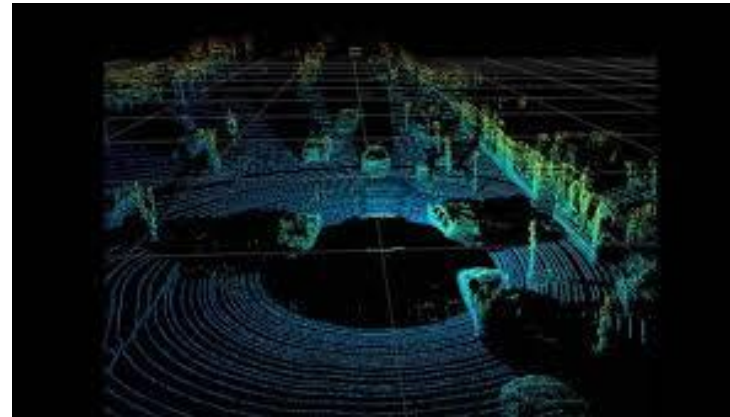
1. What is SLAM
2. The localization problem
- 3. The mapping problem**
4. Doing altogether
5. Planning while mapping
6. Nav2 ROS2 library



(3/6) The mapping problem

How to store a map ?

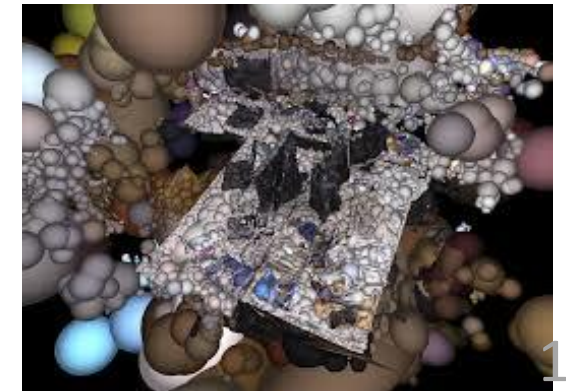
- Point cloud (from LiDAR)
- Grid cells, variable size ?
- Line map (frontiers only)
- Topological
- ...



To which cost ?

- Think about memory complexity
- How much is it needed to be twice more precise?
- $\mathcal{O}(n^2)$ or $\mathcal{O}(n^3)$ for the grid cell map ...
- Can map 10km² with 10cm precision with float 32 :
4010⁹ : 40 gb
- Can't map 10 km³ with 10 cm precision, with float 32 :
10¹⁵ bits = 125 terabytes

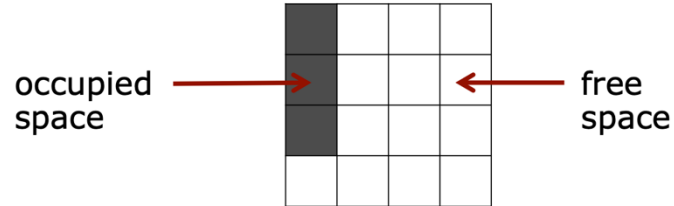
**Might need large RAM
And GPU**



(3/6) The mapping problem

How to construct a map ?

Just use your current position !

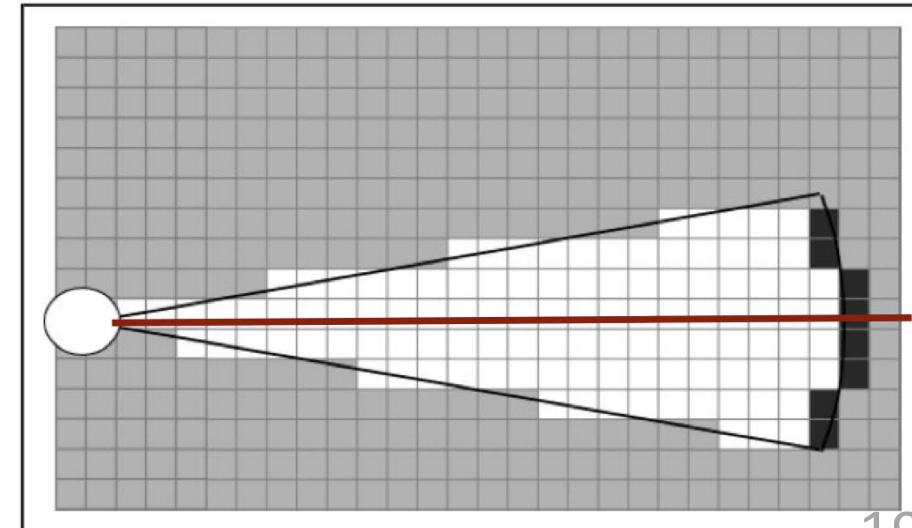
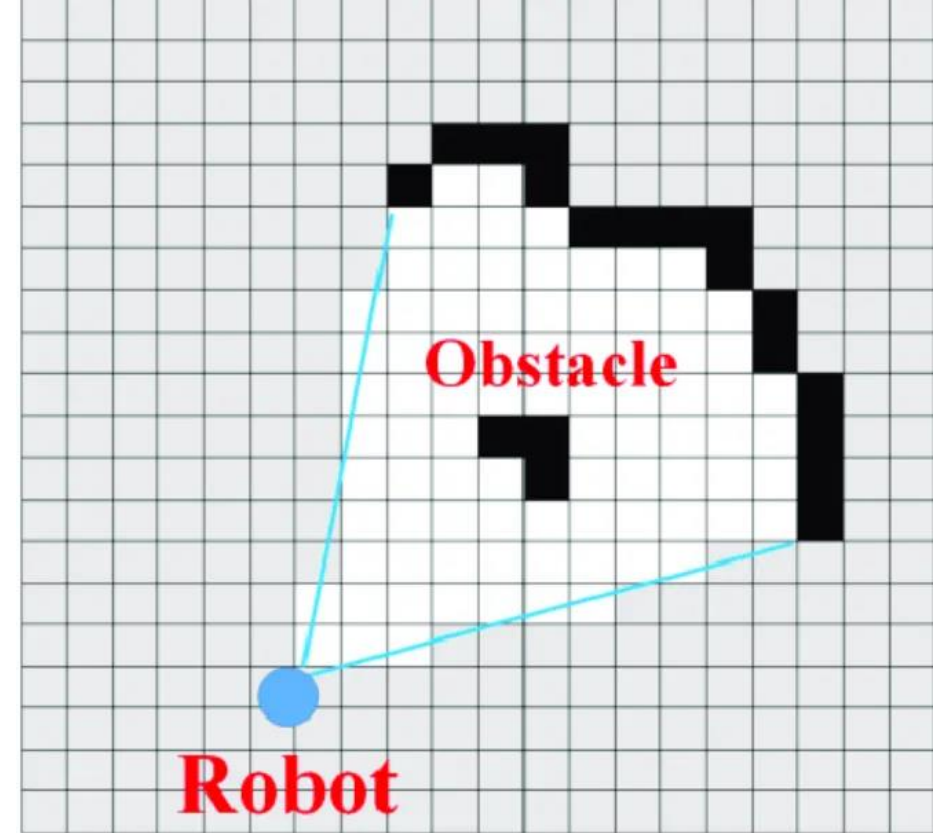


In the case of grid map :

$$(x, y)_{robot} + d(x, y) \Rightarrow \begin{cases} 1 & \text{for occupied} \\ 0 & \text{for empty cell} \end{cases}$$

We can also store if a cell has been seen or not, or last time it has been seen...

Source :
[Uni-Freiburg](#)



(3/6) The mapping problem

How to construct a map ?

But, is this enough ?

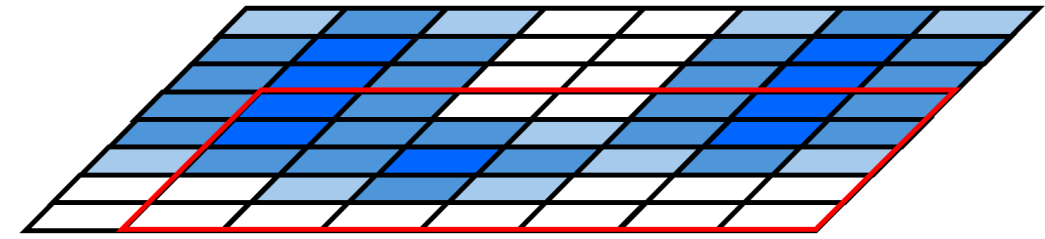
Some problems :

- What if an obstacle is half cell?

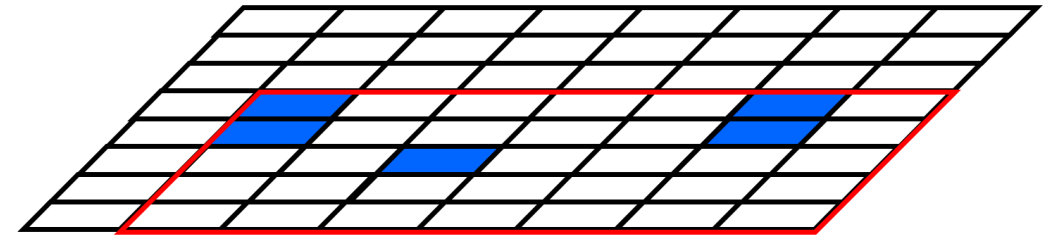
⇒ **Inflated cost map**

- What if we have a wrong measurement or if the environment is evolving?

⇒ **Probabilities**

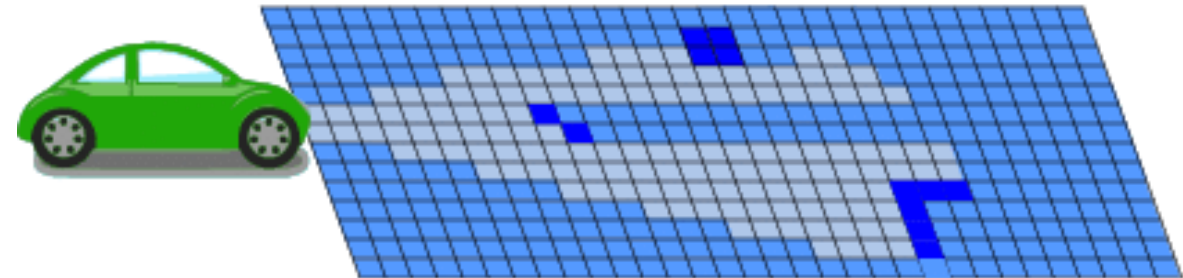


Inflation Layer



Obstacle Map Layer

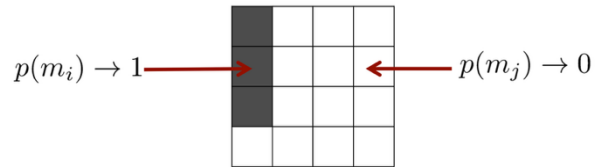
Occupancy Probability: $p \rightarrow 0$  $p \sim 0.5$  $p \rightarrow 1$ 



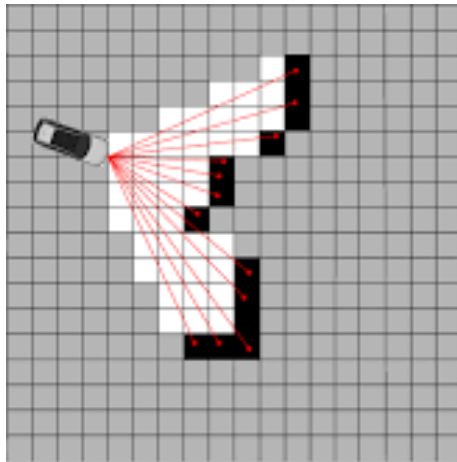
(3/6) The mapping problem

How to construct a map **with probabilities** ?

- Each cell is a **binary random variable** that models the occupancy

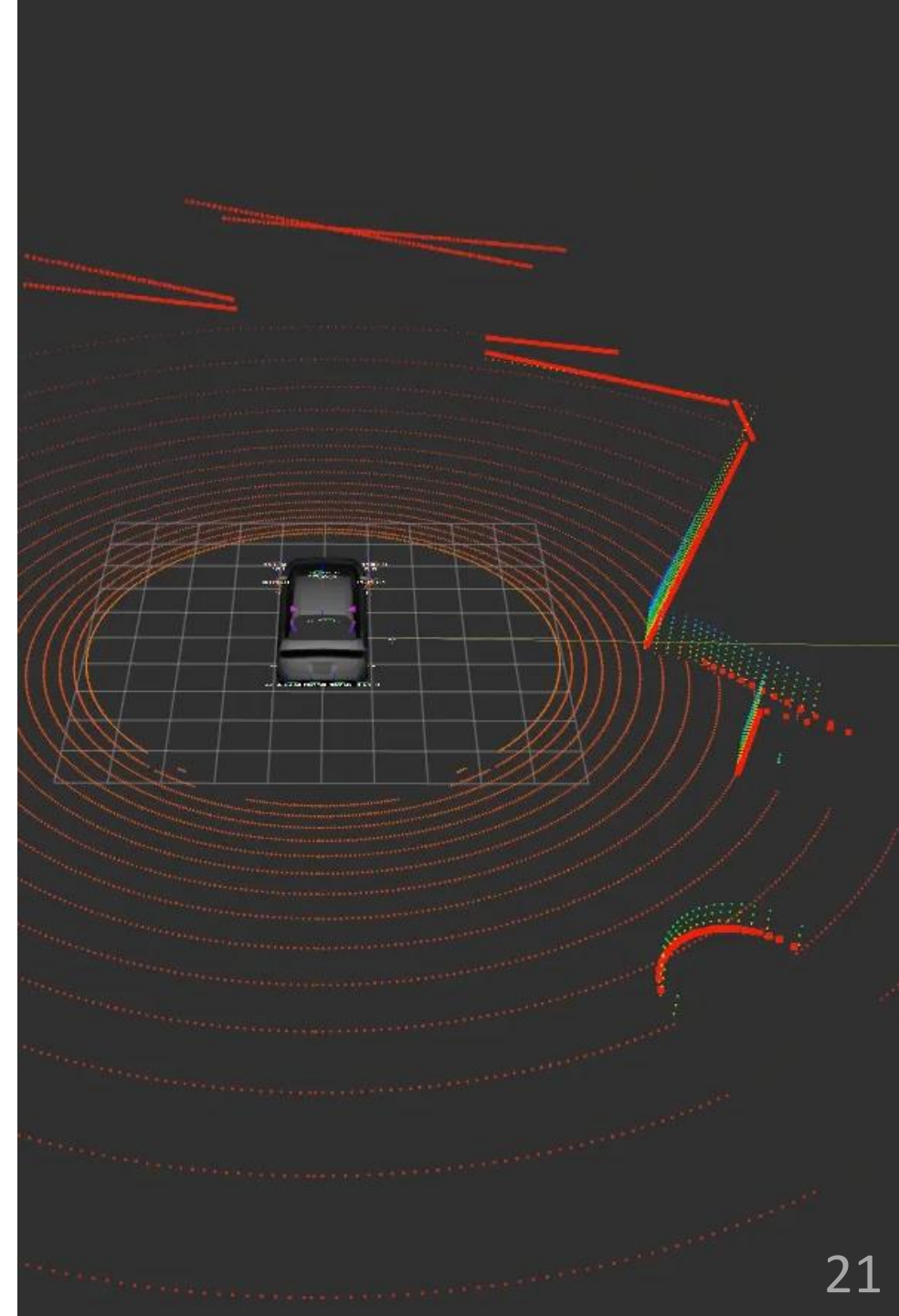


- Each cell is a **binary random variable** that models the occupancy
- Cell is occupied: $p(m_i) = 1$
- Cell is not occupied: $p(m_i) = 0$
- No knowledge: $p(m_i) = 0.5$



- $p_{z_{k+1}}(O_{k+1}|z_{k+1}) = 0.95$
- $p_{z_{k+1}}(O_{k+1}|z_{k+1}) = 0.05$
- $p_{z_{k+1}}(O_{k+1}|z_{k+1}) = 0.5$

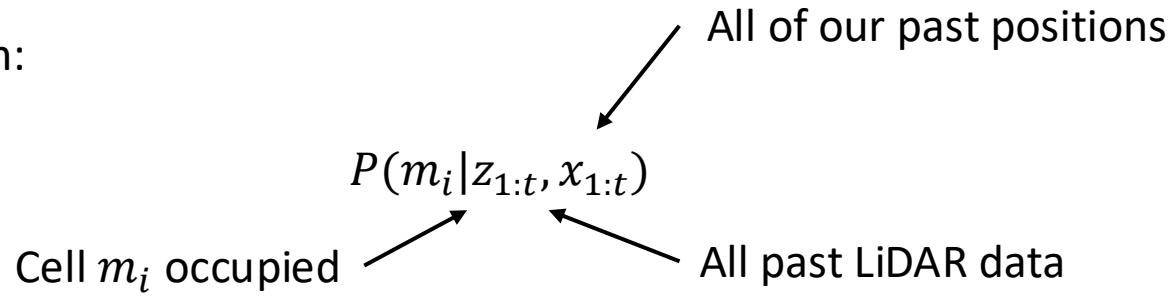
Source :
[Uni-Freiburg](http://www.uni-freiburg.de)



(3/6) The mapping problem

How to update a probability map ?

Our problem:

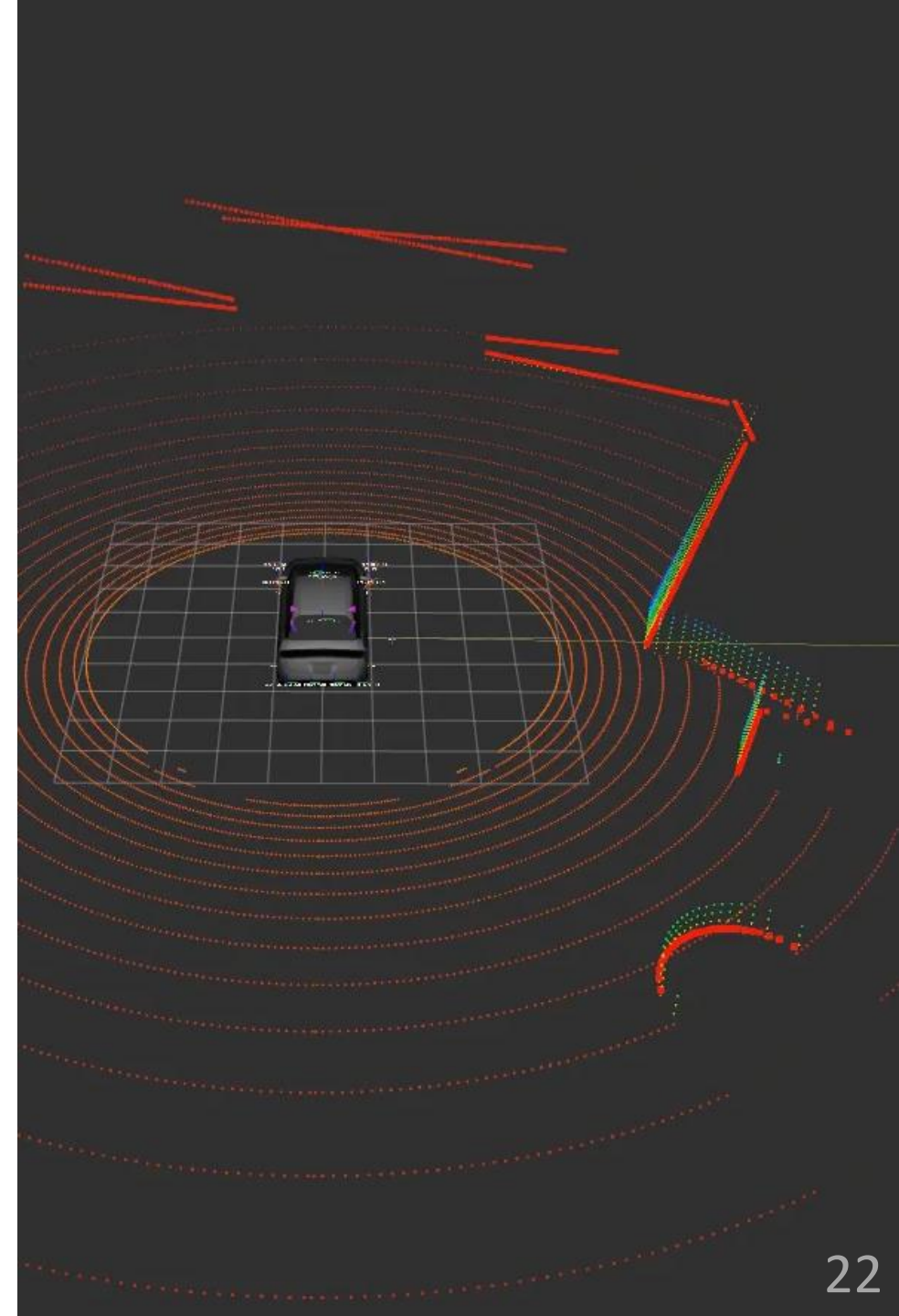


The main idea, state is known, use Baye's rule to update:

$$P(m_i | z_{1:t}) = \frac{P(z_t | m_i) P(m_i | z_{1:t-1})}{P(z_t)}$$

Source :

[Uni-Freiburg](#)



(3/6) The mapping problem

The main idea, use Baye's rule:

$$P(m_i | z_{1:t}) = \frac{P(z_t | m_i)}{P(z_t)} P(m_i | z_{1:t-1})$$

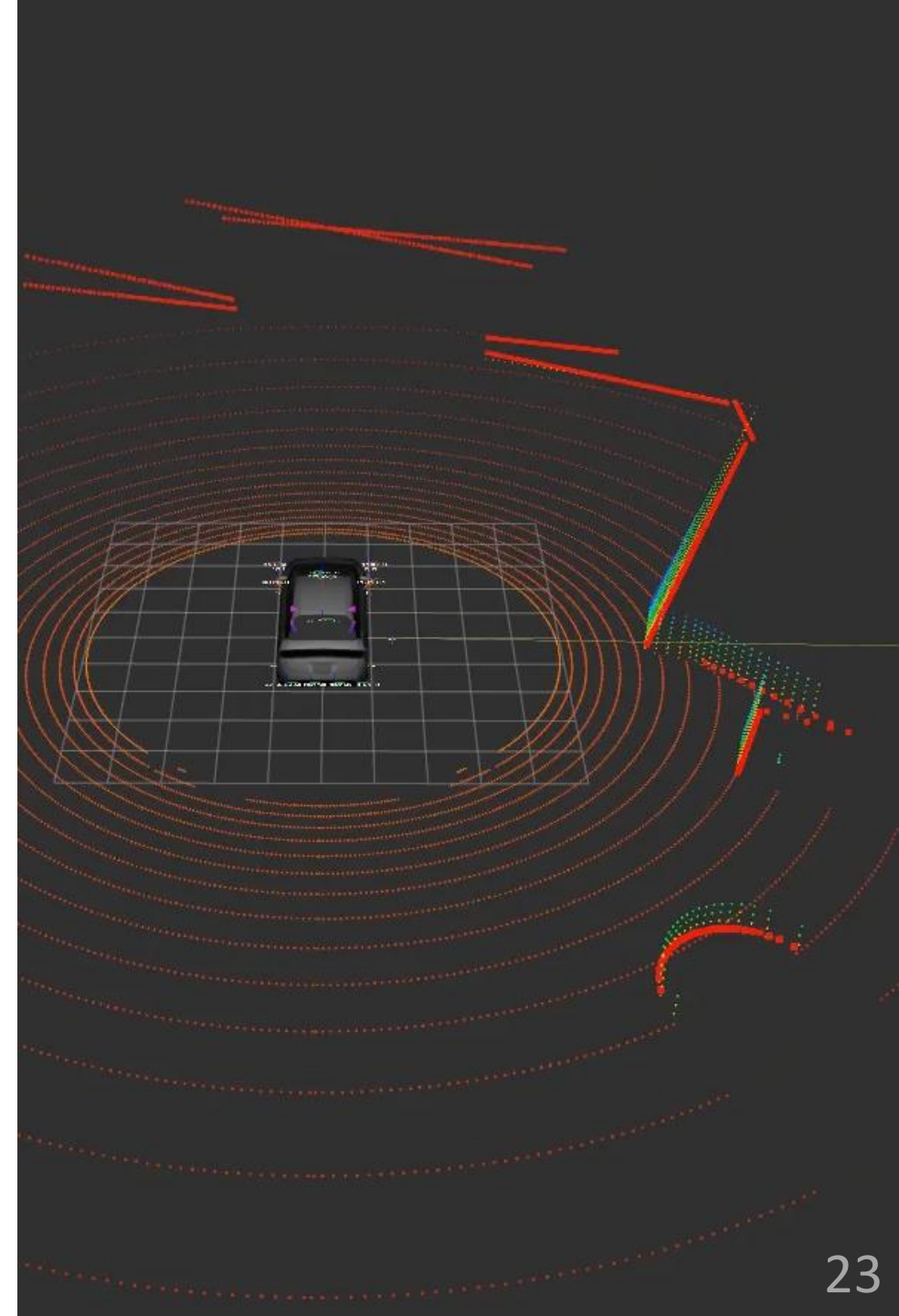
Posterior belief of being occupied Probability of the measurement, **constant** Prior belief of being occupied

Case 1:



Sensor locates something :

$$\begin{aligned} P(z_t | occupied) &\gg P(z_t | free) \\ P(z_t | m_i) &> 1 \\ \Rightarrow P(m_i | z_{1:t}) &> P(m_i | z_{1:t-1}) \end{aligned}$$



(3/6) The mapping problem

The main idea, use Baye's rule:

$$P(m_i|z_{1:t}) = \frac{P(z_t|m_i)}{P(z_t)} P(m_i|z_{1:t-1})$$

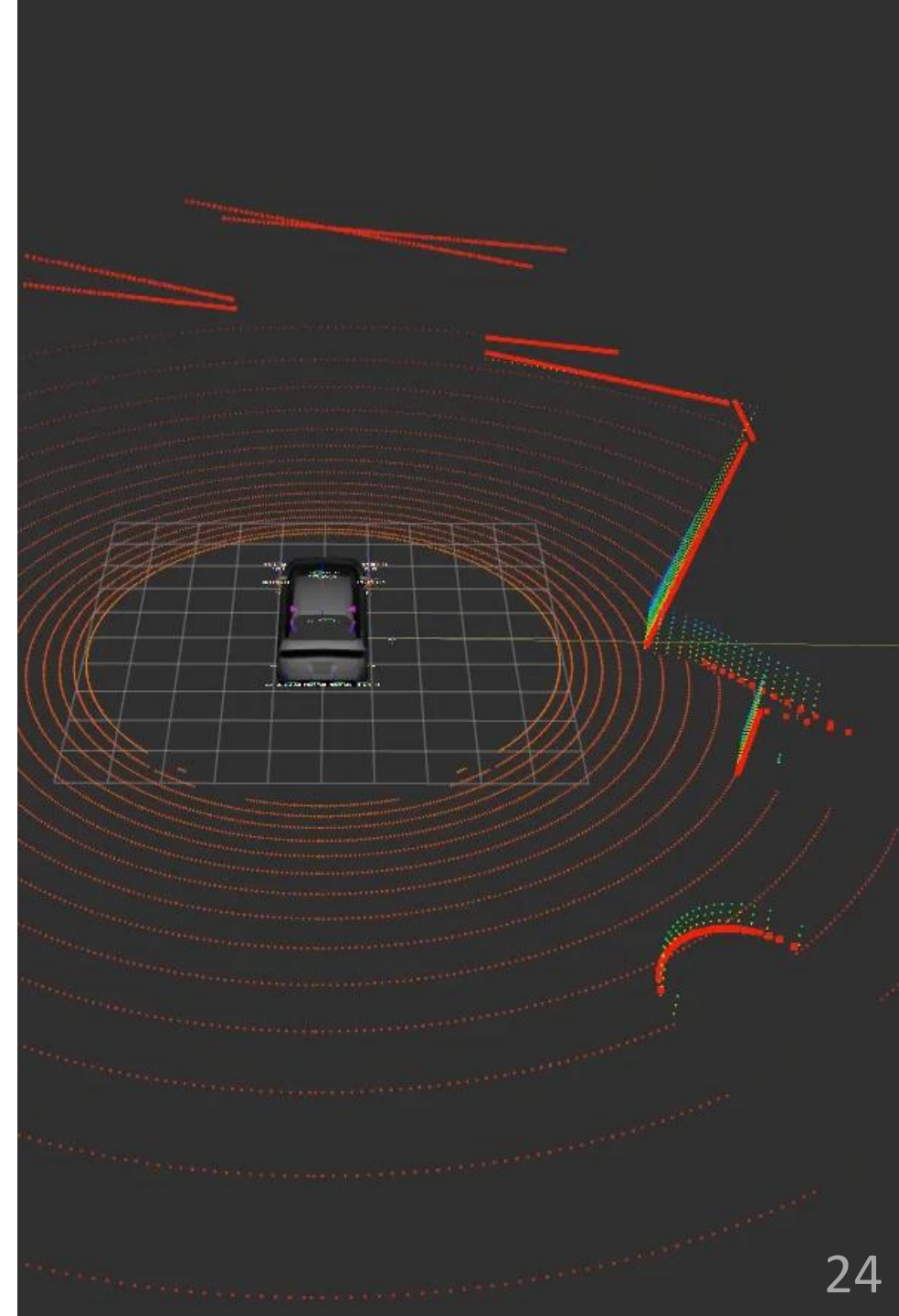
Posterior belief of being occupied Probability of the measurement, **constant** Prior belief of being occupied

Case 1:



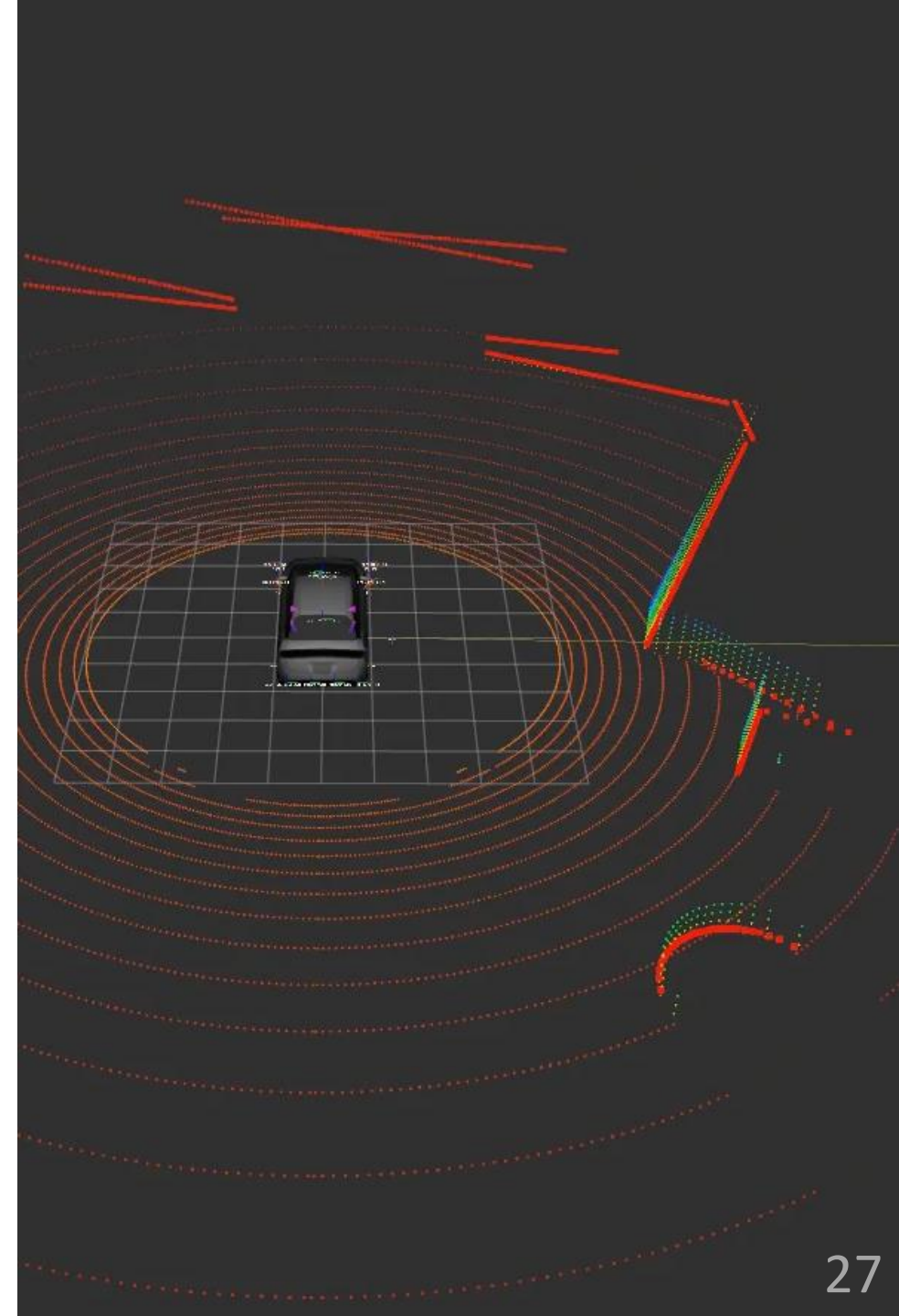
Sensor beam passes through :

$$\begin{aligned} P(z_t | free) &\gg P(z_t | occupied) \\ P(z_t|m_i) &< 1 \\ \Rightarrow P(m_i|z_{1:t}) &< P(m_i|z_{1:t-1}) \end{aligned}$$



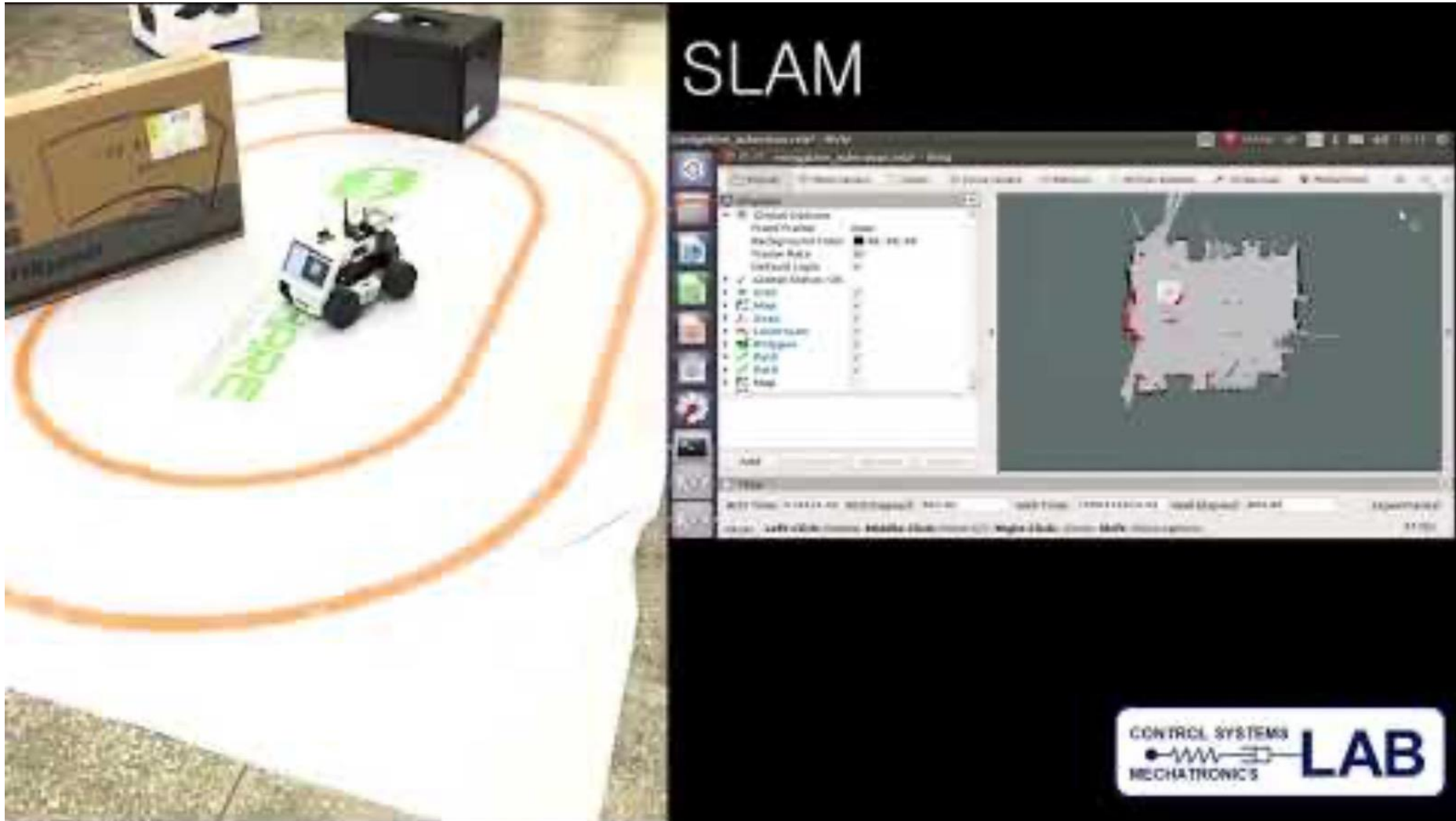
Plan

1. What is SLAM
2. The localization problem
3. The mapping problem
- 4. Doing altogether**
5. Planning while mapping
6. Nav2 ROS2 library



(4/6) Doing altogether

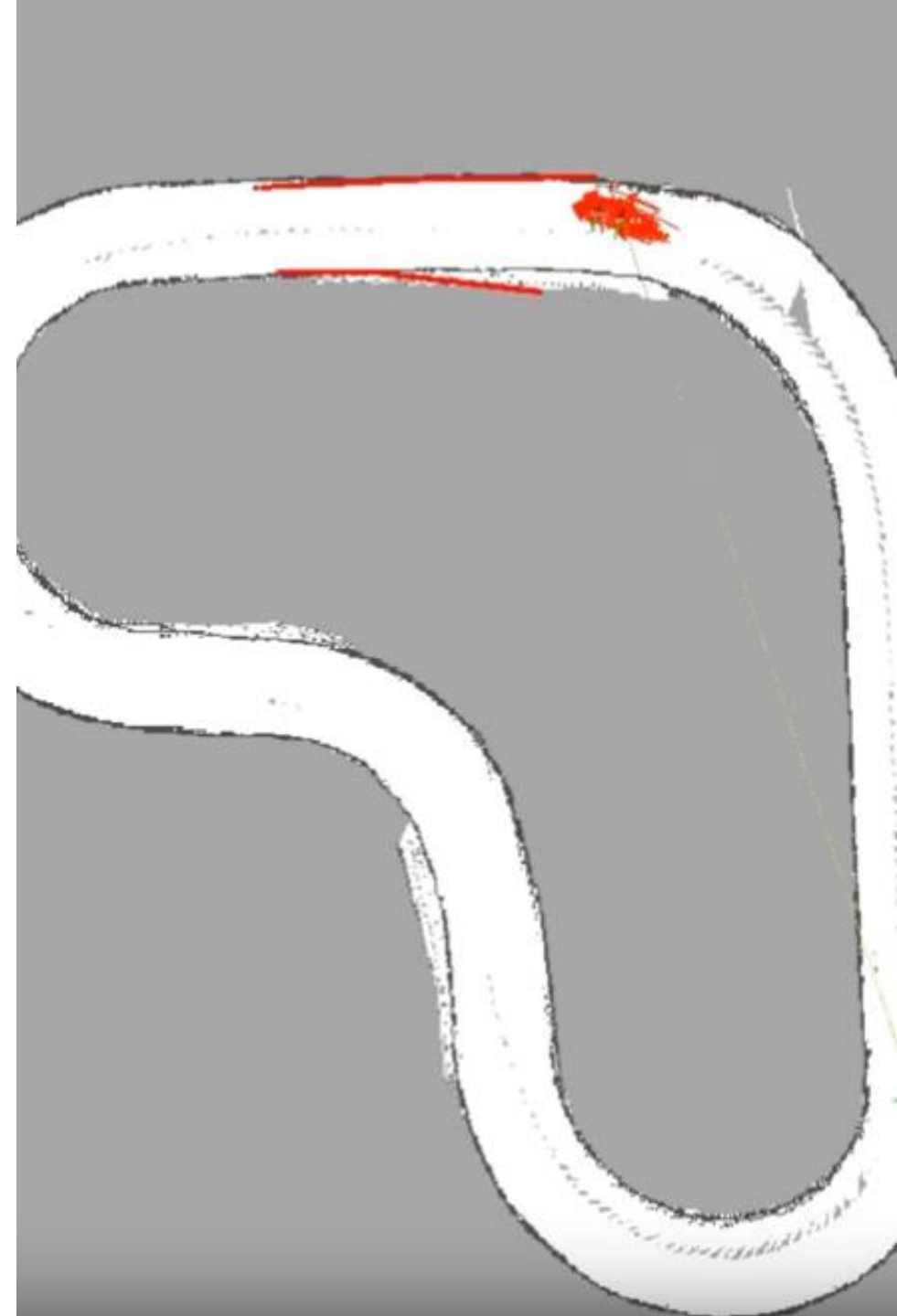
Updating the map from current movement :
Looks to how boundaries are updated



(4/6) Doing altogether

How can you map your environment the first time?

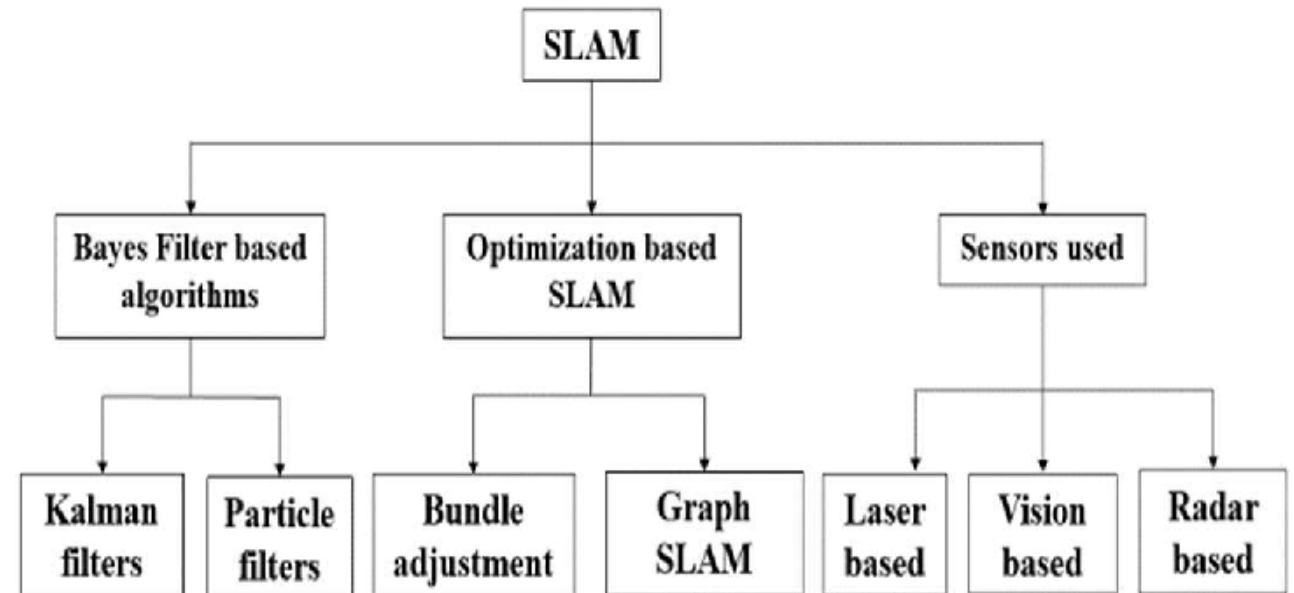
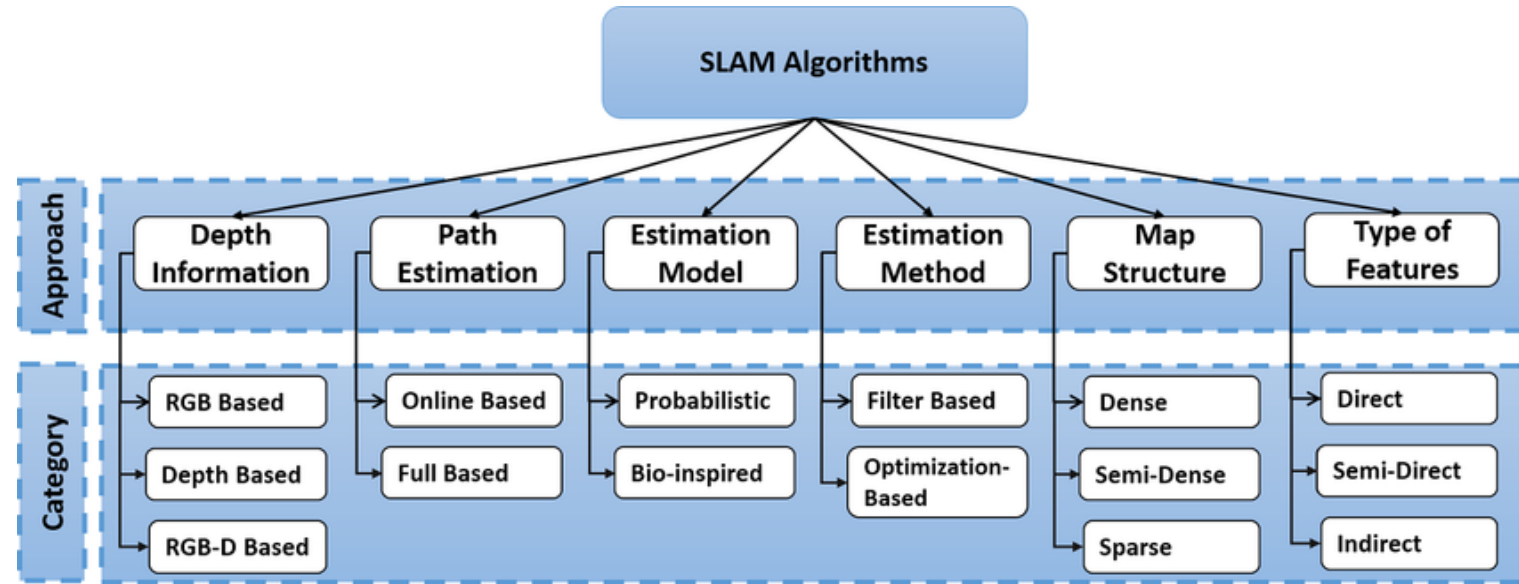
- **Remote control ?**
 - Yes, but not what we want 😊
- **Random walk ?**
 - Guarantee to work when $t \rightarrow +\infty$!
- **Naive approaches ? – requires prior knowledge**
 - Follow the wall
 - Follow the average direction of the wall
 - ...
- **Geometric coverage?**
 - Linear movements
 - Circular/Spiral movements
 - Map every cell one by one, why not ...
 - ...



(4/6) Doing altogether

Many existing improvements/variations :

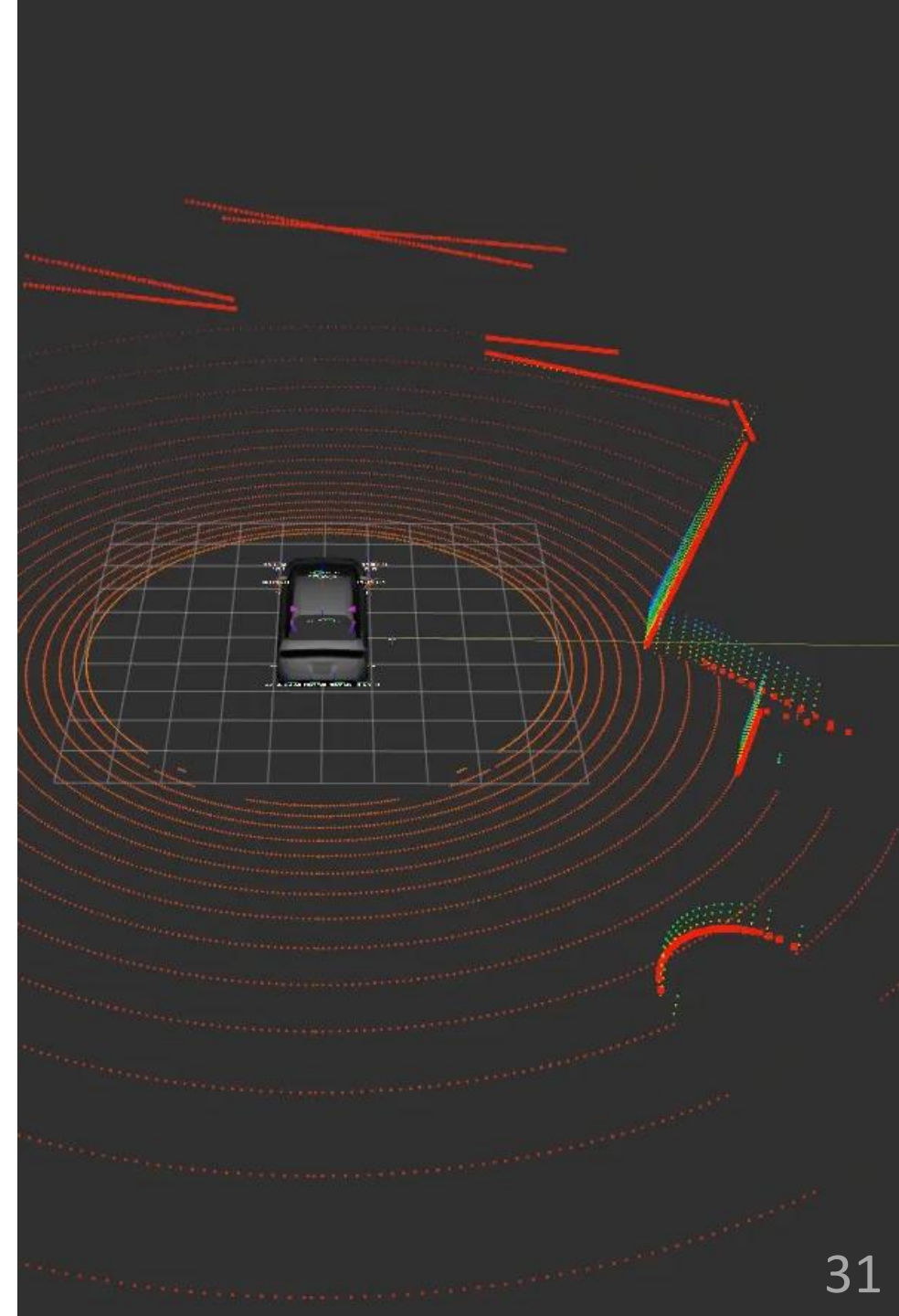
- EKF SLAM
- Visual SLAM: monocular/binocular
- Graph-based SLAM
- Semantic SLAM
- Dynamic SLAM
- 3D SLAM
- Learning-based SLAM
- ...



Plan

1. What is SLAM
2. The localization problem
3. The mapping problem
4. Doing altogether
- 5. Planning while mapping**
6. Nav2 ROS2 library

**Short
Break time** 



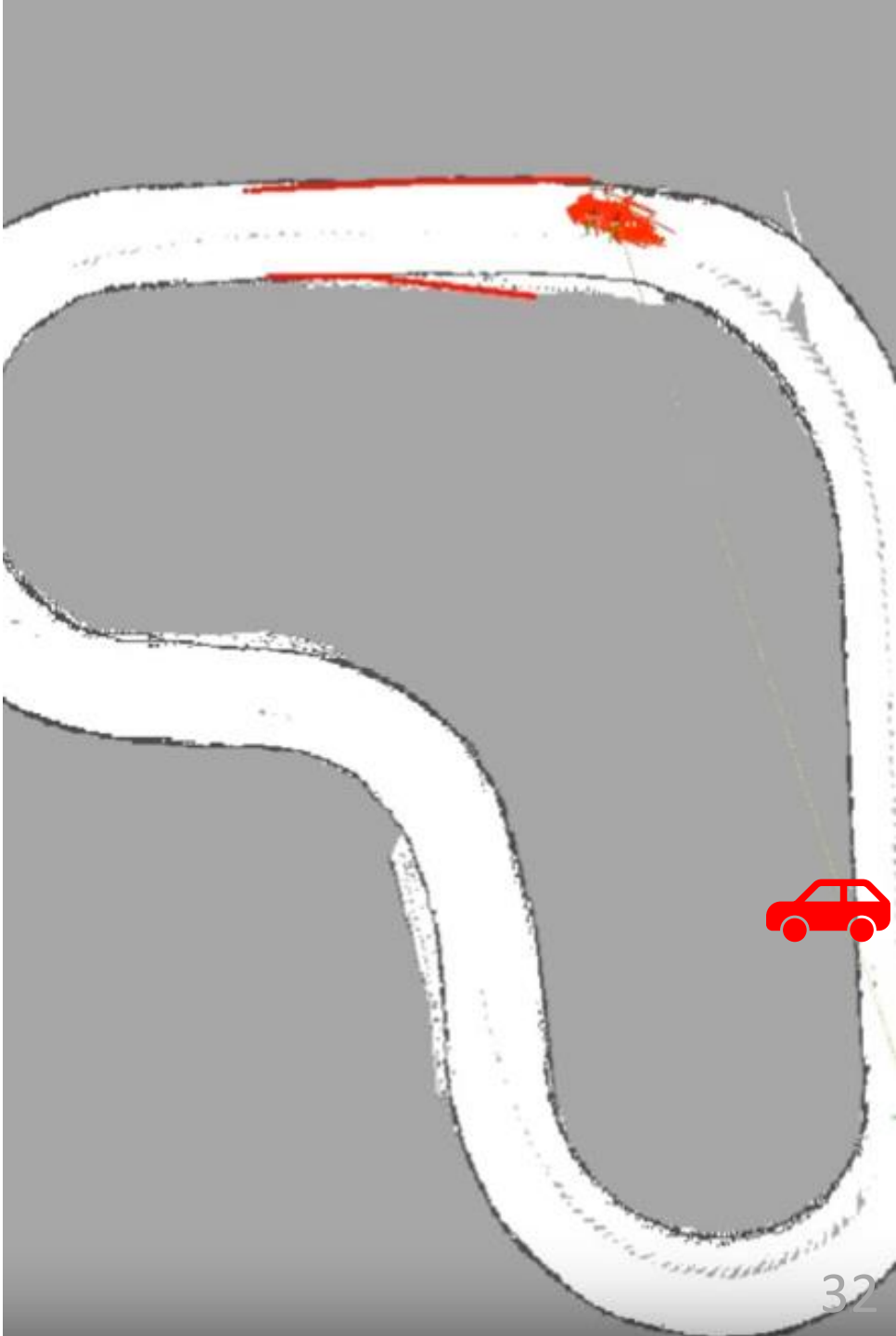
(5/6) Planning while mapping

Step 1

We can perform trajectory computation while mapping !

Illustrative example

				0.6	0.6	0.6
				0.2	0.2	0.6
				0.6	0.2	0.6
				0.6	0.2	0.6
				0.7	X	0.7
				0.7	Virtual wall	1.0
					Goal	



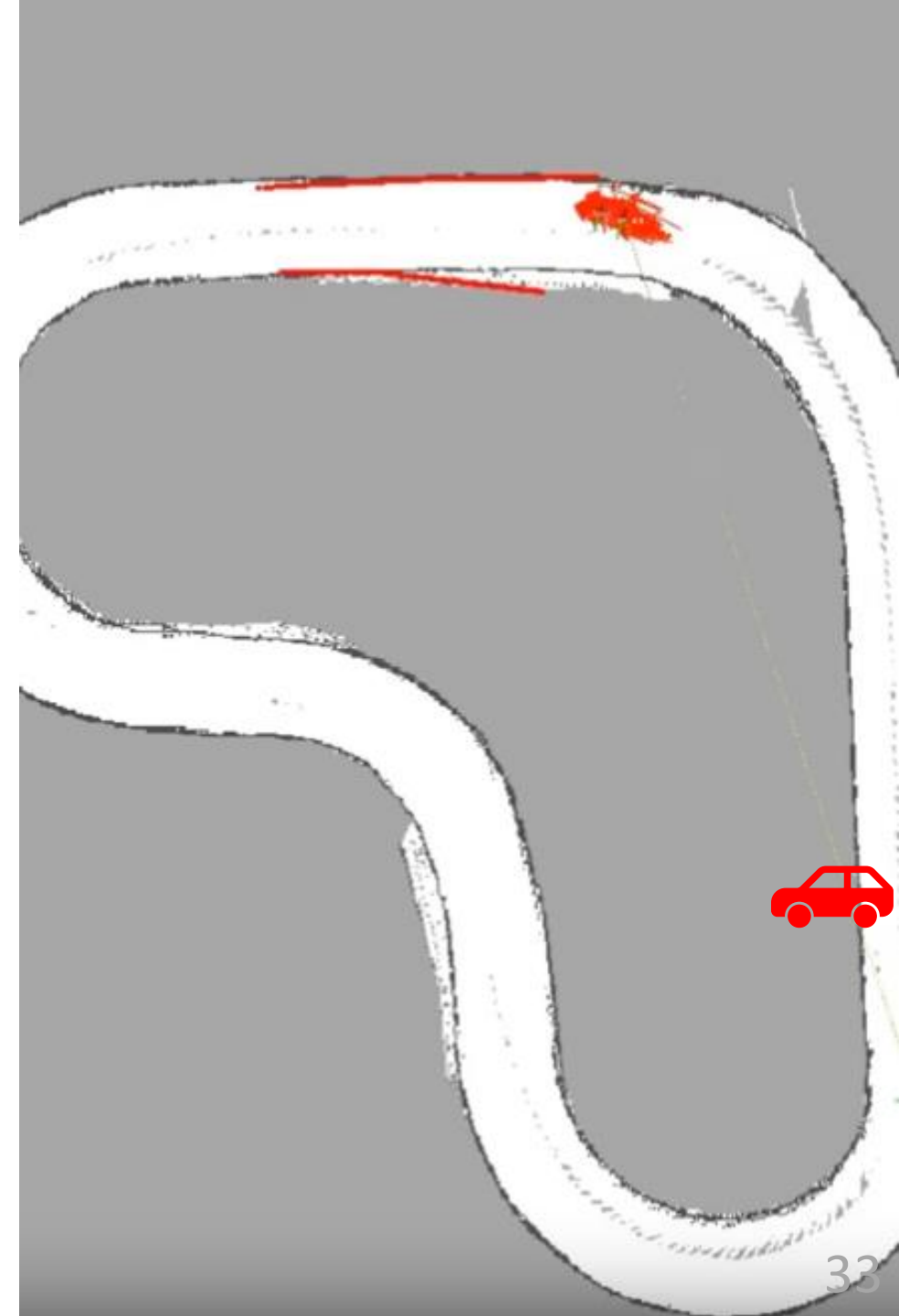
(5/6) Planning while mapping

Step 1

We can perform trajectory computation while mapping !

Illustrative example

				0.6	0.6	0.6
				0.2	0.2	0.6
				0.6	0.2	0.6
				0.6	0.2	0.6
				0.7	X	0.7
				0.7	Virtual wall	1.0
					Goal	



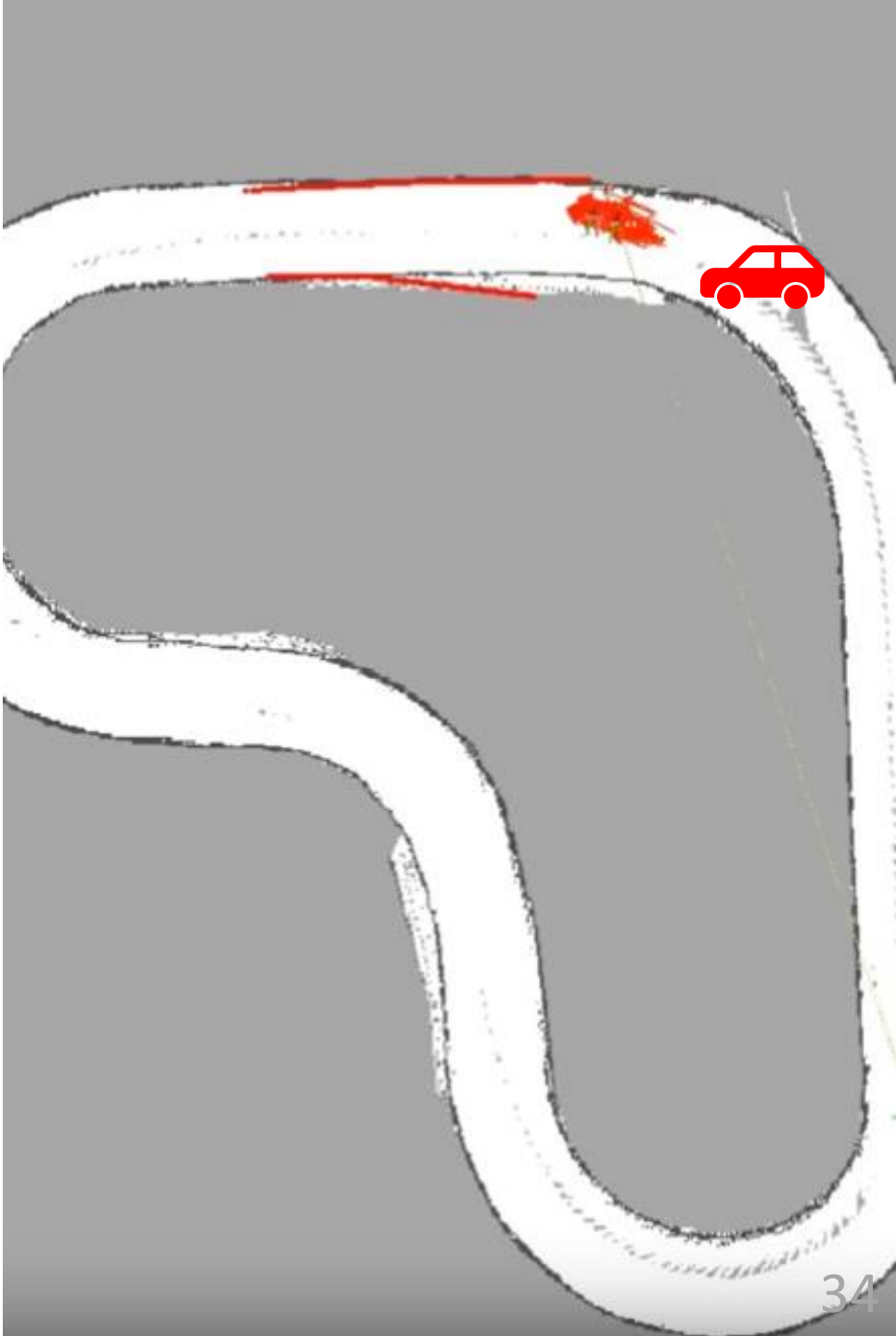
(5/6) Planning while mapping

Step 2

We can perform trajectory computation while mapping !

Illustrative example

?	?	?	0.7	1.0	1.0	1.0
0.7	0.3	0.3	0.2	0.1	X	1.0
?	?	?	0.7	1.0	0.0	1.0
				1.0	0.0	1.0
				1.0	0.0	1.0
				1.0	Virtual wall	1.0
					Goal	



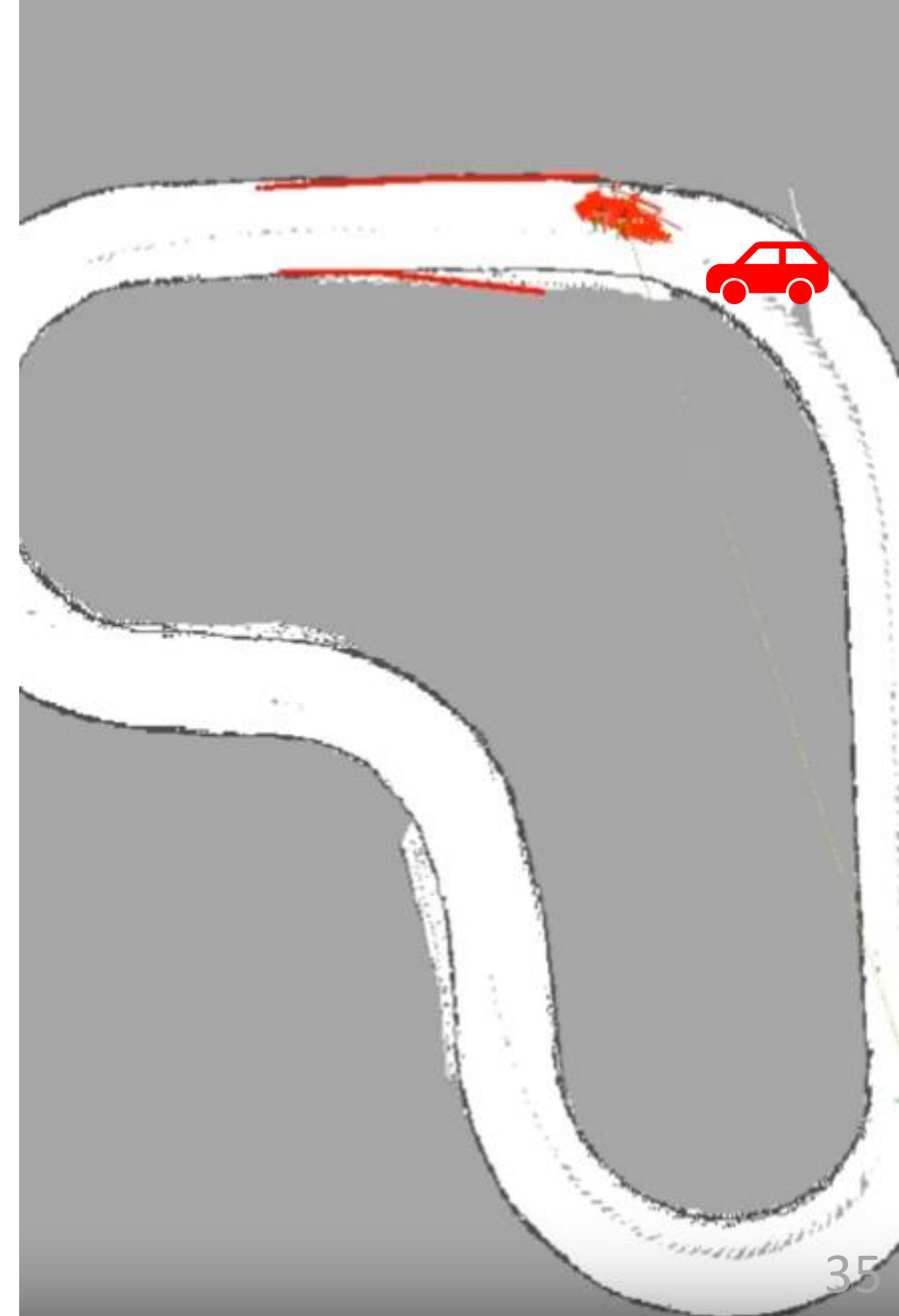
(5/6) Planning while mapping

Step 2

We can perform trajectory computation while mapping !

Illustrative example

?	?	?	0.7	1.0	1.0	1.0
0.7	0.3	0.3	0.2	0.1	X	1.0
?	?	?	0.7	1.0	0.0	1.0
				1.0	0.0	1.0
				1.0	0.0	1.0
				1.0	Virtual wall	1.0
					Goal	



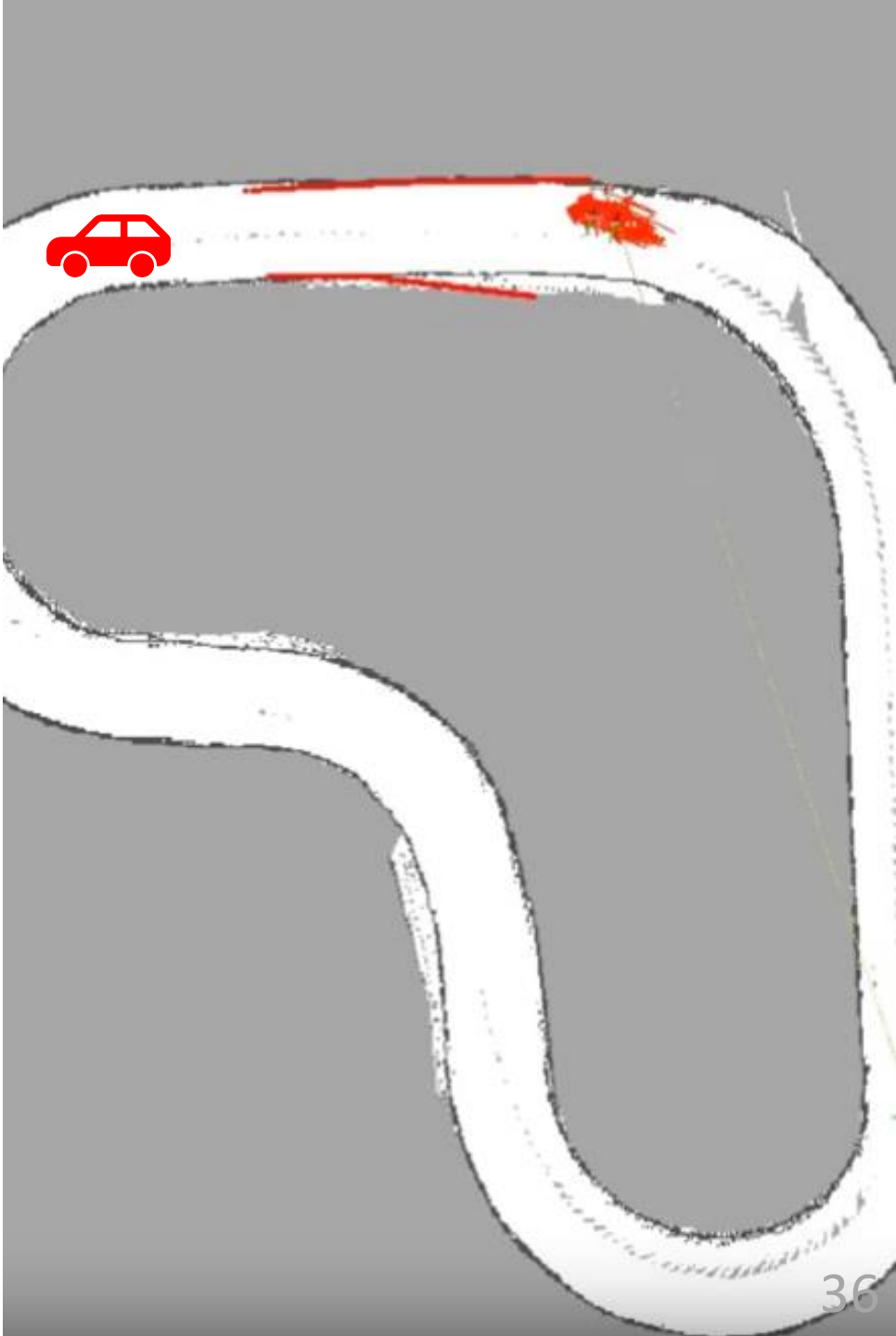
(5/6) Planning while mapping

Step 2

We can perform trajectory computation while mapping !

Illustrative example

1.0	1.0	1.0	1.0	1.0	1.0	1.0
1.0	X	0.0	0.0	0.0	0.0	1.0
1.0	0.2	1.0	1.0	1.0	0.0	1.0
0.6	0.3	0.6		1.0	0.0	1.0
0.6	0.7	0.6		1.0	0.0	1.0
				1.0	Virtual wall	1.0
					Goal	



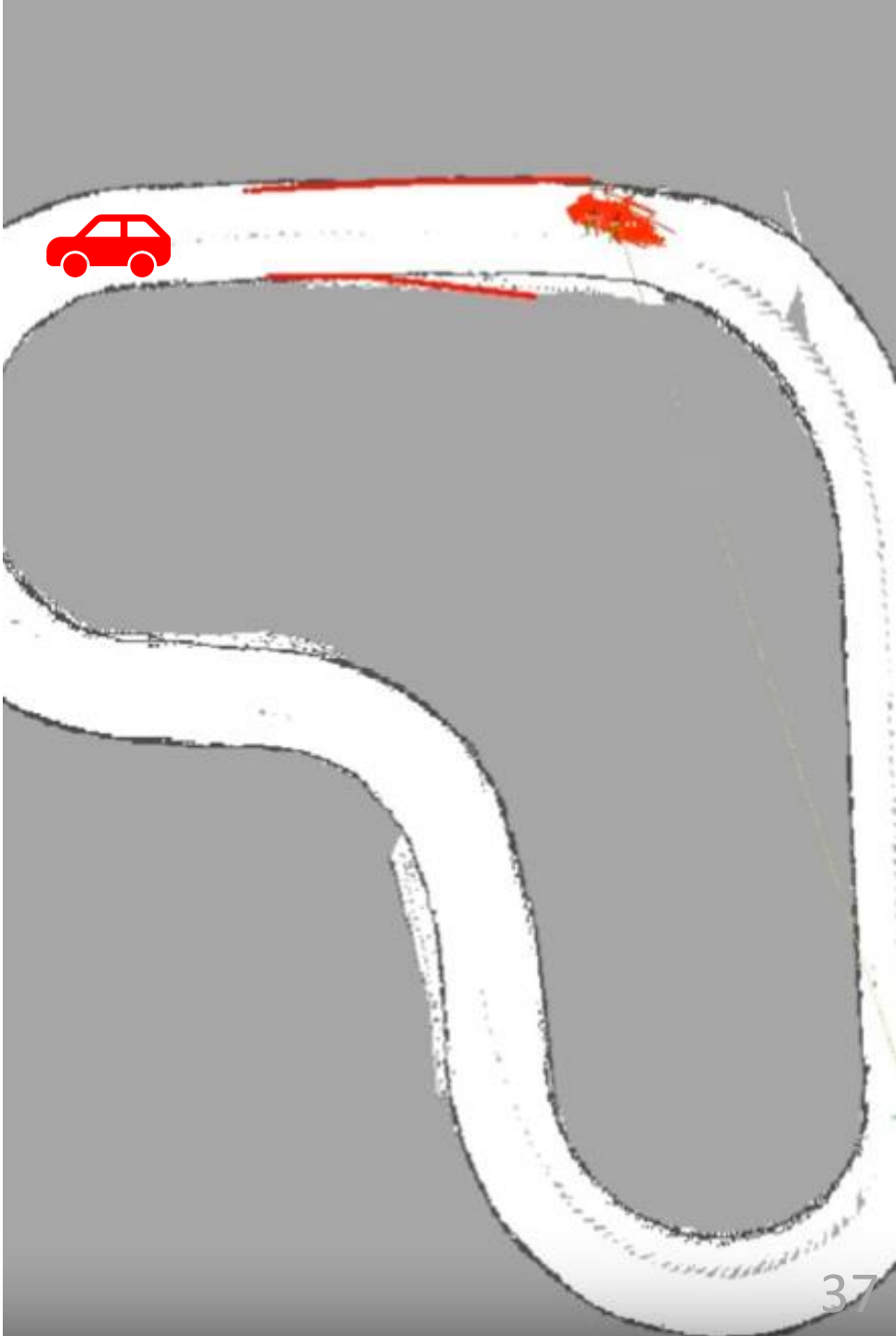
(5/6) Planning while mapping

Step 2

We can perform trajectory computation while mapping !

Illustrative example

1.0	1.0	1.0	1.0	1.0	1.0	1.0
1.0	X	0.0	0.0	0.0	0.0	1.0
1.0	0.2	1.0	1.0	1.0	0.0	1.0
0.6	0.3	0.6		1.0	0.0	1.0
0.6	0.7	0.6		1.0	0.0	1.0
				1.0	Virtual wall	1.0
					Goal	



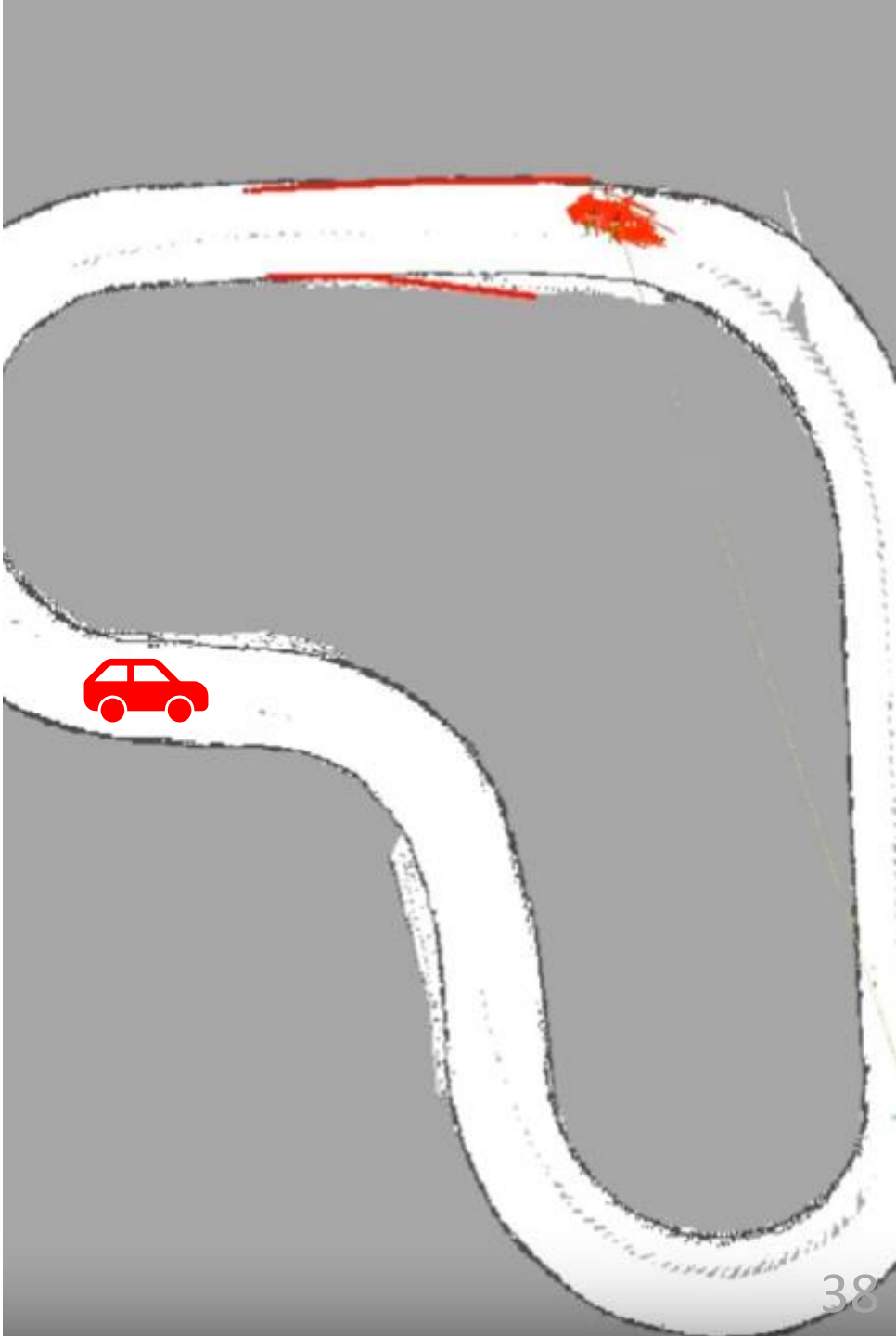
(5/6) Planning while mapping

Step 3

We can perform trajectory computation while mapping !

Illustrative example

1.0	1.0	1.0	1.0	1.0	1.0	1.0
1.0	0.0	0.0	0.0	0.0	0.0	1.0
1.0	0.0	1.0	1.0	1.0	0.0	1.0
1.0	X	0.2	0.2	1.0	0.0	1.0
1.0	1.0	1.0		1.0	0.0	1.0
				1.0	Virtual wall	1.0
					Goal	



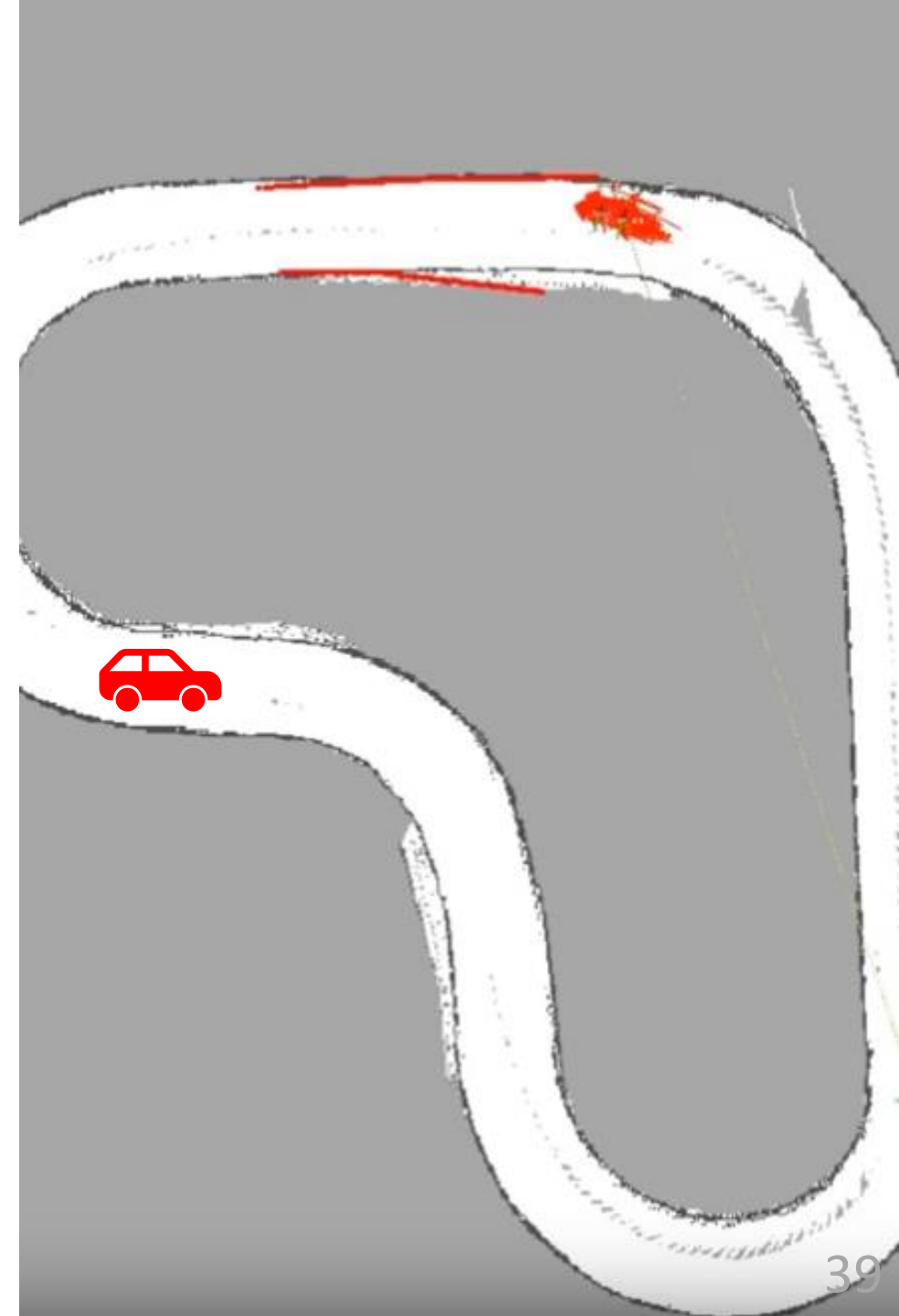
(5/6) Planning while mapping

Step 3

We can perform trajectory computation while mapping !

Illustrative example

1.0	1.0	1.0	1.0	1.0	1.0	1.0
1.0	0.0	0.0	0.0	0.0	0.0	1.0
1.0	0.0	1.0	1.0	1.0	0.0	1.0
1.0	X	0.2	0.2	1.0	0.0	1.0
1.0	1.0	1.0		1.0	0.0	1.0
				1.0	Virtual wall	1.0
					Goal	



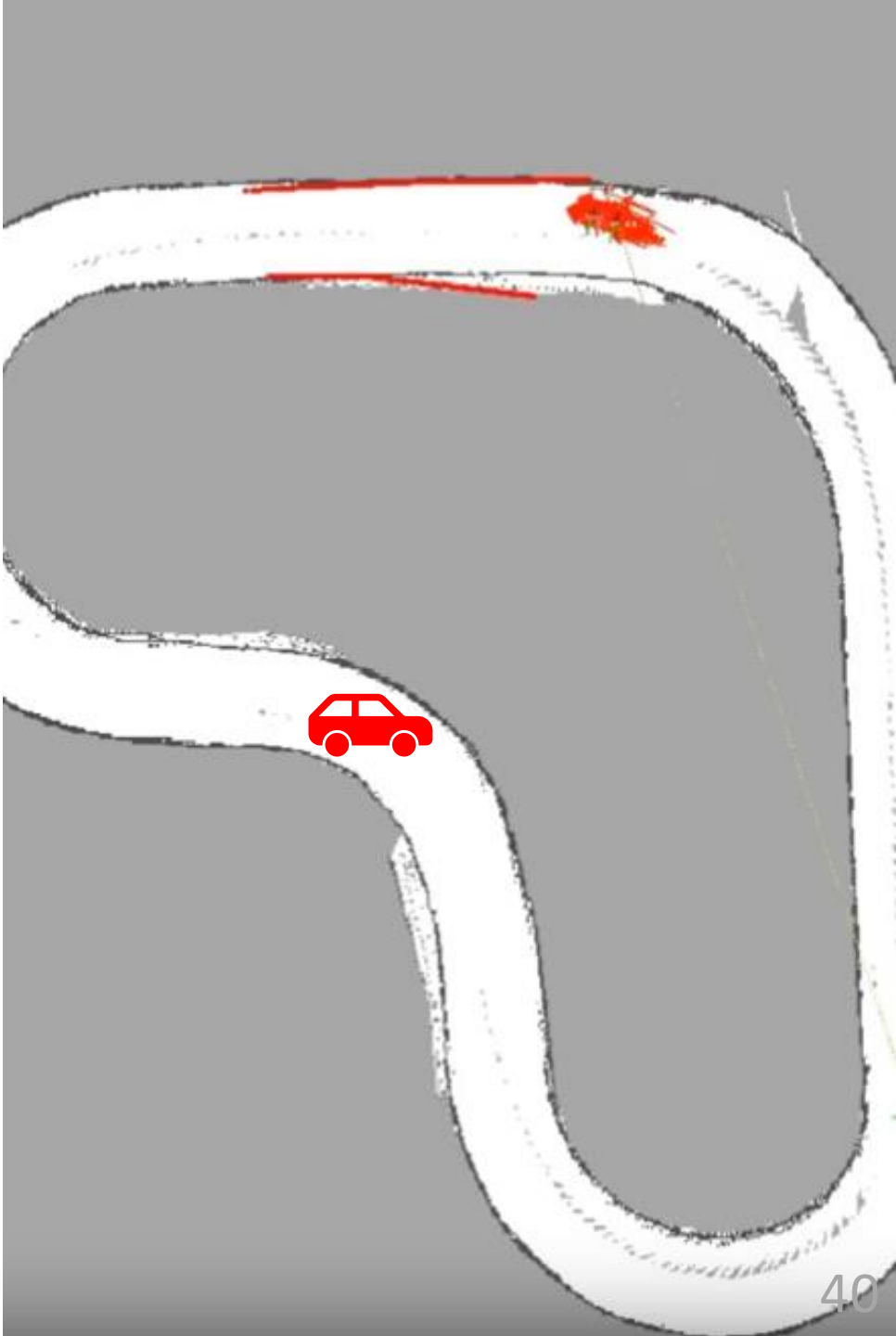
(5/6) Planning while mapping

Step 4

We can perform trajectory computation while mapping !

Illustrative example

1.0	1.0	1.0	1.0	1.0	1.0	1.0
1.0	0.0	0.0	0.0	0.0	0.0	1.0
1.0	0.0	1.0	1.0	1.0	0.0	1.0
1.0	0.0	0.0	X	1.0	0.0	1.0
1.0	1.0	1.0	0.2	1.0	0.0	1.0
		0.7	0.2	1.0	Virtual wall	1.0
		0.6	0.3		Goal	
			0.6			



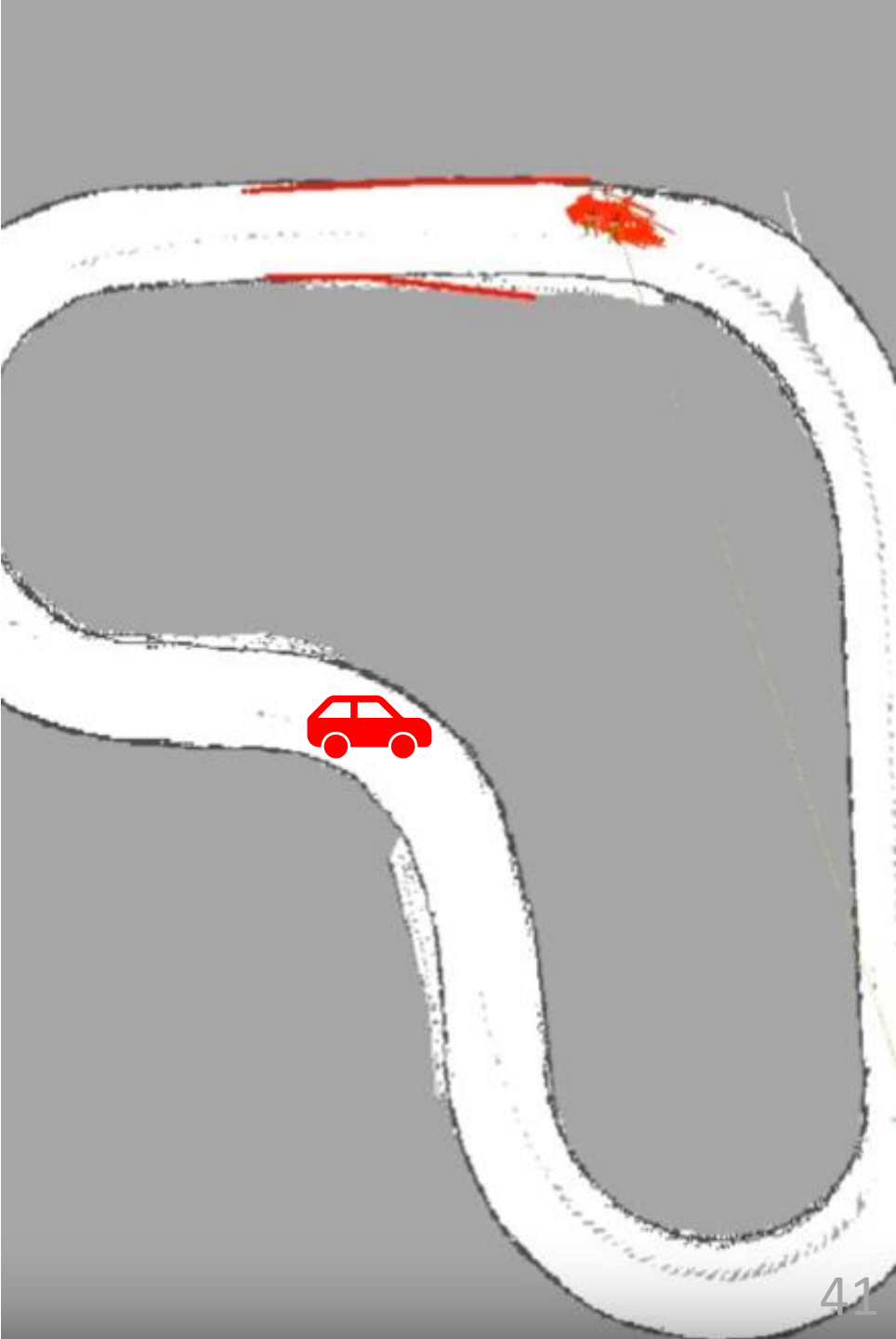
(5/6) Planning while mapping

Step 4

We can perform trajectory computation while mapping !

Illustrative example

1.0	1.0	1.0	1.0	1.0	1.0	1.0
1.0	0.0	0.0	0.0	0.0	0.0	1.0
1.0	0.0	1.0	1.0	1.0	0.0	1.0
1.0	0.0	0.0	X	1.0	0.0	1.0
1.0	1.0	1.0	0.2	1.0	0.0	1.0
		0.7	0.1	1.0	Virtual wall	1.0
		0.6	0.3	Goal		
			0.6			



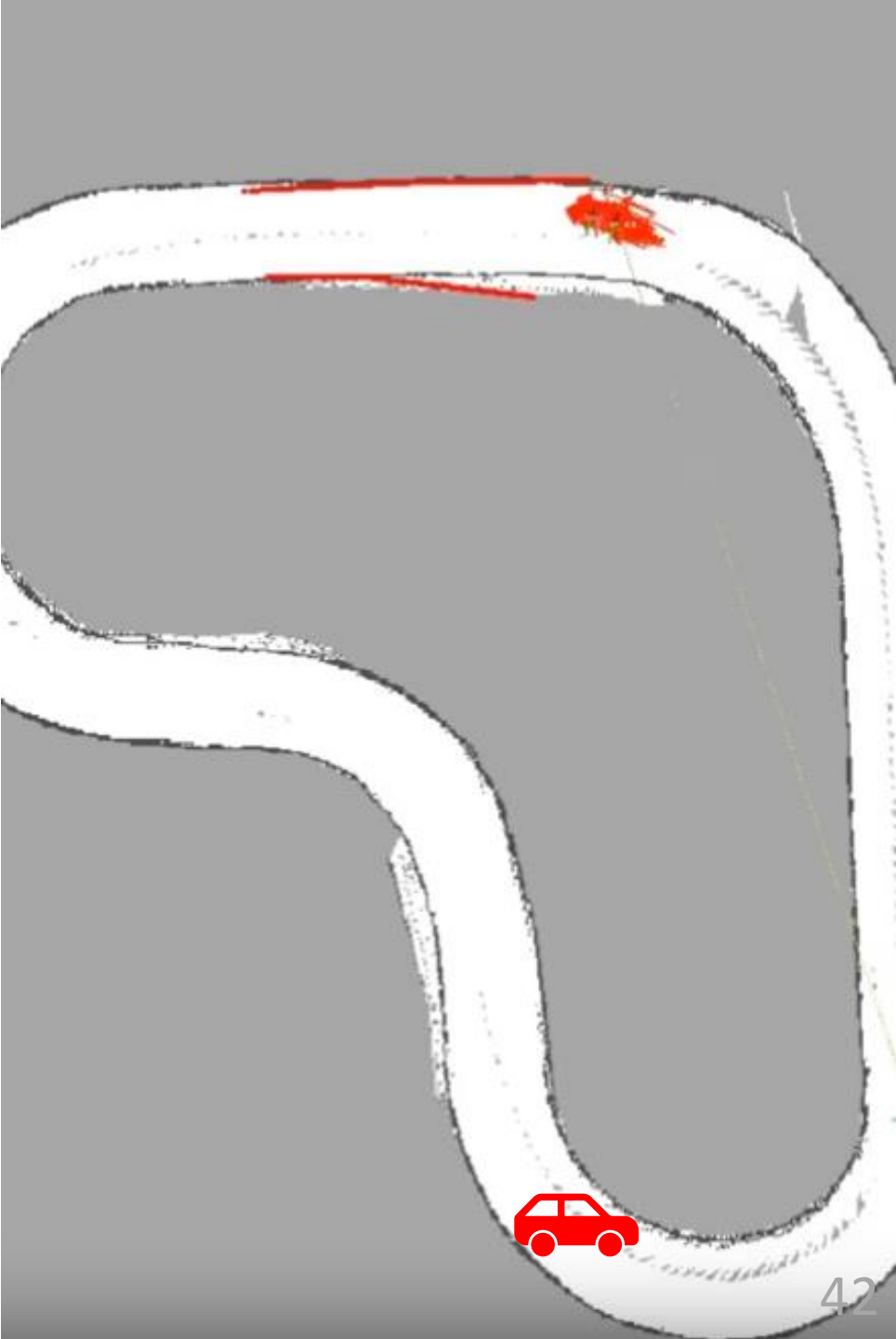
(5/6) Planning while mapping

Step 5

We can perform trajectory computation while mapping !

Illustrative example

1.0	1.0	1.0	1.0	1.0	1.0	1.0
1.0	0.0	0.0	0.0	0.0	0.0	1.0
1.0	0.0	1.0	1.0	1.0	0.0	1.0
1.0	0.0	0.0	0.0	1.0	0.0	1.0
1.0	1.0	1.0	0.0	1.0	0.0	1.0
		1.0	0.0	1.0	Virtual wall	1.0
		1.0	X	0.6	Goal	0.7
		1.0	1.0	0.7	0.7	0.7



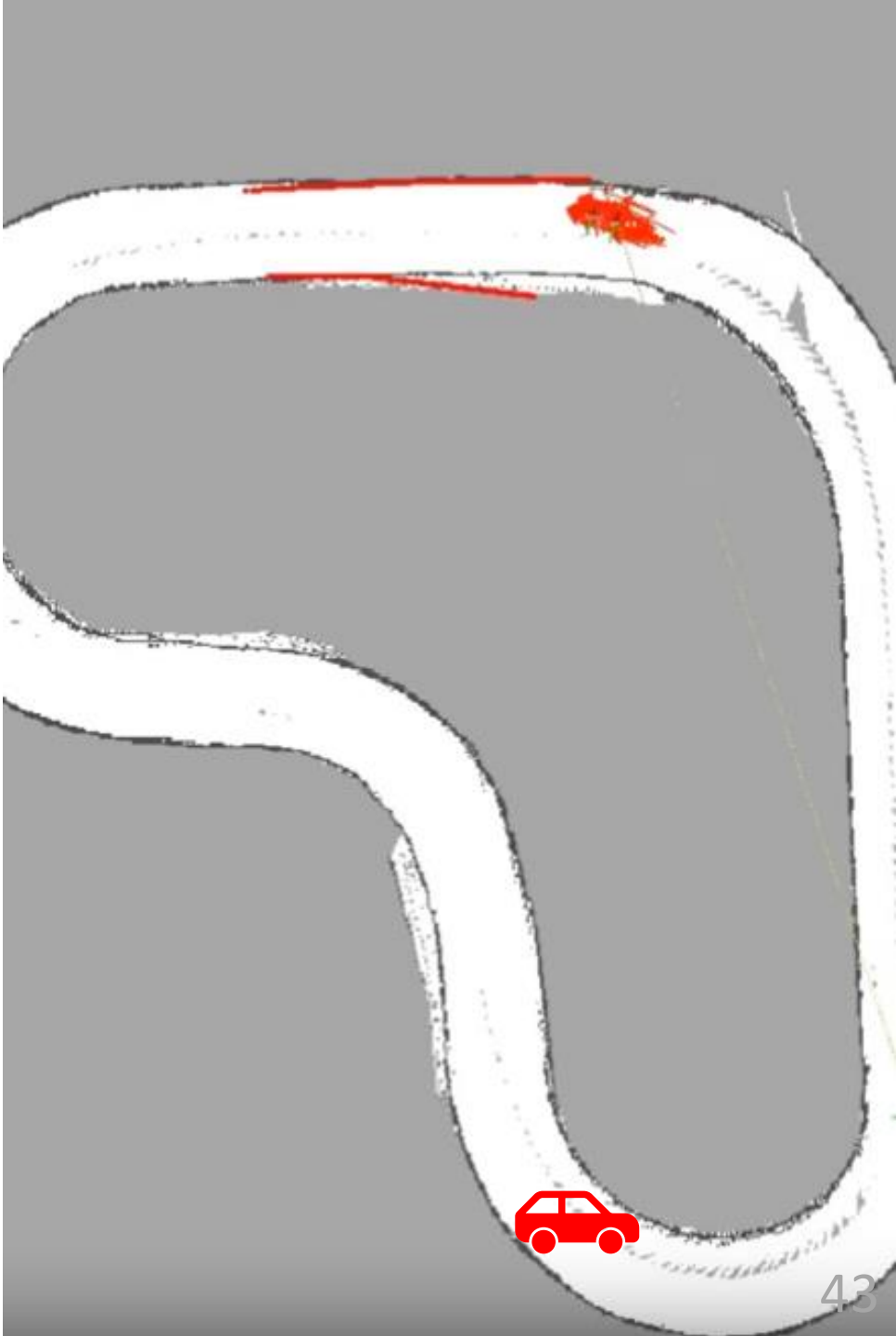
(5/6) Planning while mapping

Step 5

We can perform trajectory computation while mapping !

Illustrative example

1.0	1.0	1.0	1.0	1.0	1.0	1.0
1.0	0.0	0.0	0.0	0.0	0.0	1.0
1.0	0.0	1.0	1.0	1.0	0.0	1.0
1.0	0.0	0.0	0.0	1.0	0.0	1.0
1.0	1.0	1.0	0.0	1.0	0.0	1.0
		1.0	0.0	1.0	Virtual wall	1.0
		1.0	X	- - - 0.2 - - -	Goal	0.7
		1.0	1.0	0.7	0.7	0.7

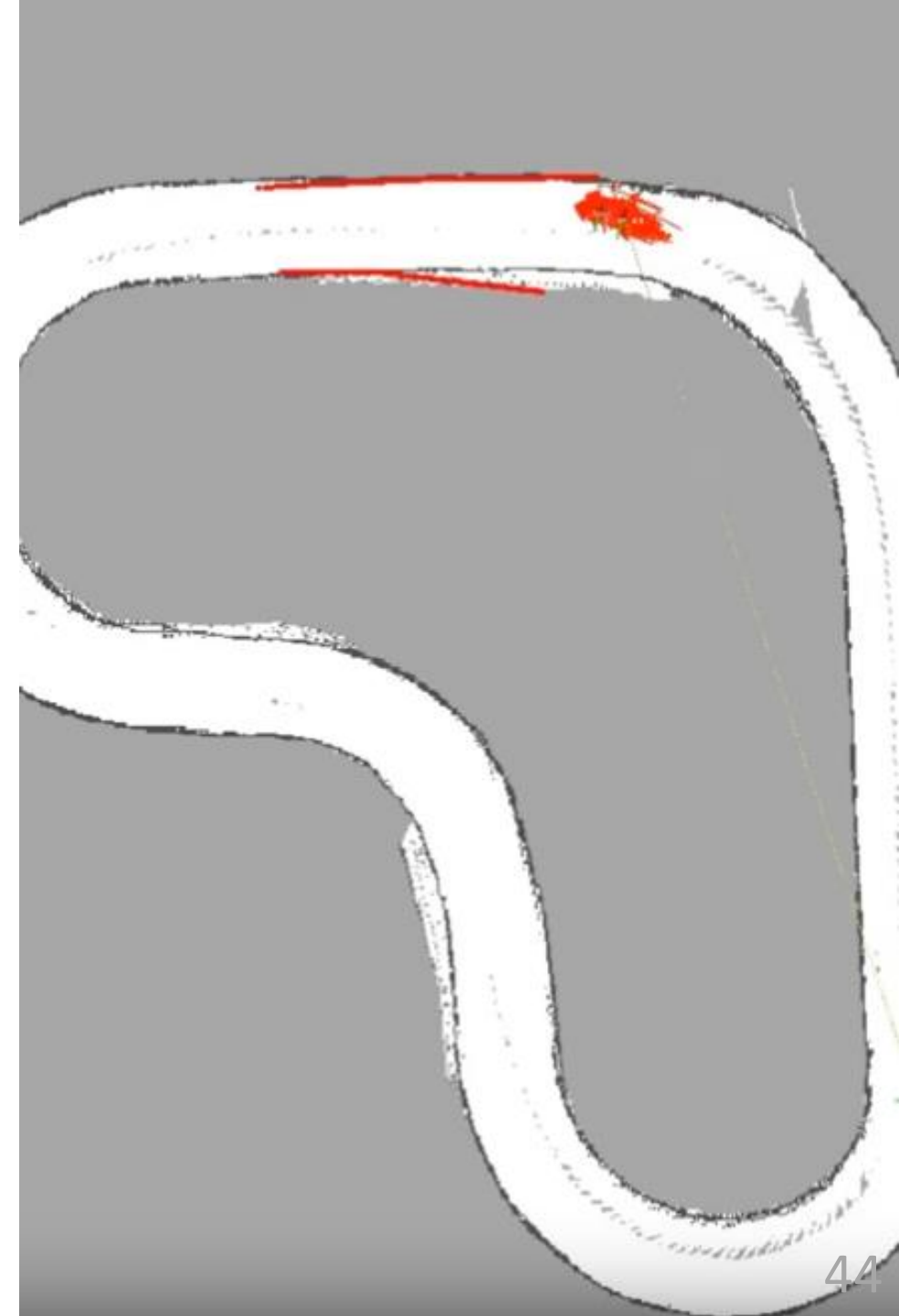


(5/6) Planning while mapping

We can perform trajectory computation while mapping !

The global solution:

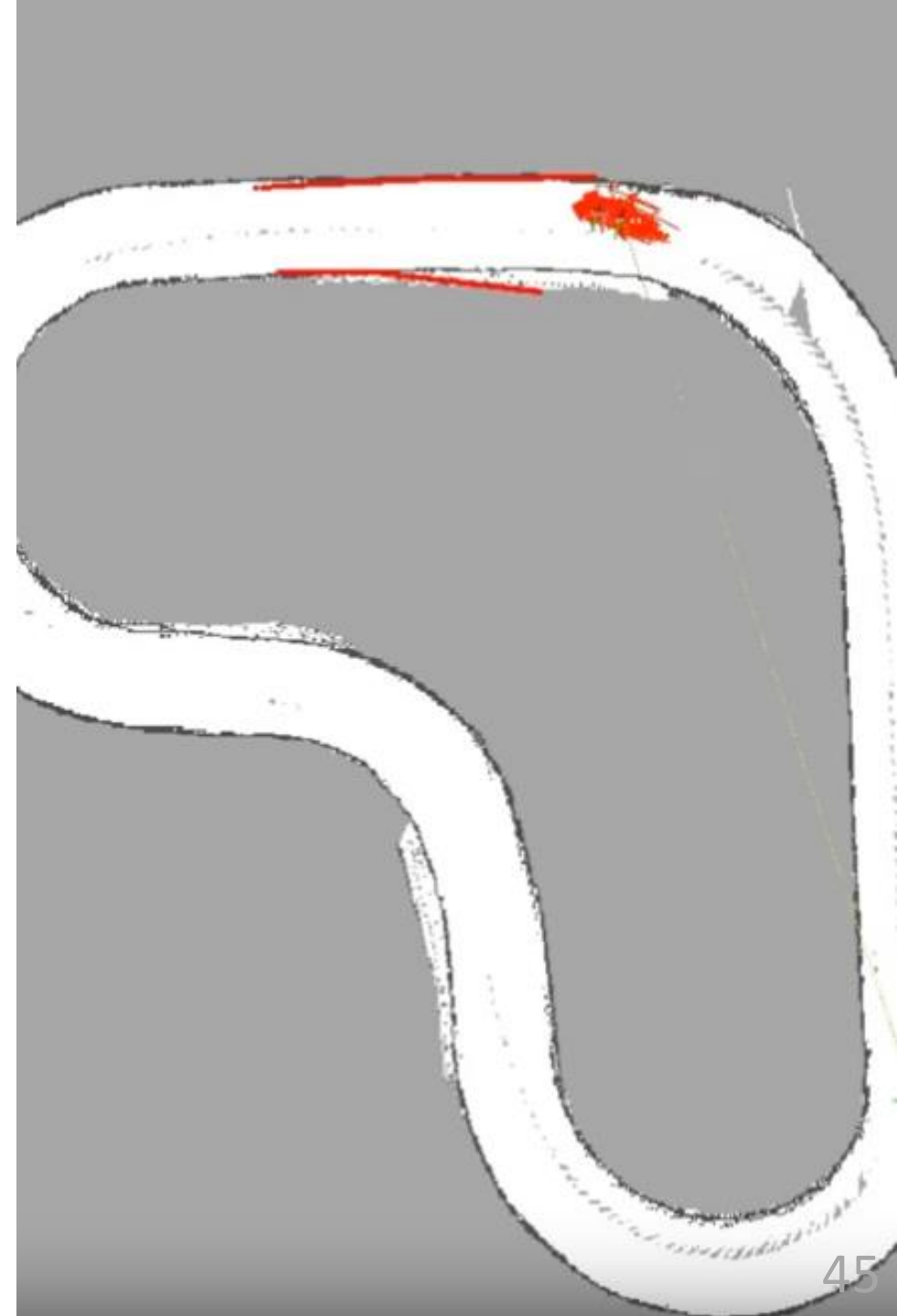
1.0	1.0	1.0	1.0	1.0	1.0	1.0
1.0						1.0
1.0		1.0	1.0	1.0		1.0
1.0				1.0		1.0
1.0	1.0	1.0		1.0	X	1.0
?	?	1.0		1.0	Virtual wall	1.0
?	?	1.0			X	1.0
?	?	1.0	1.0	1.0	1.0	1.0



(5/6) Planning while mapping

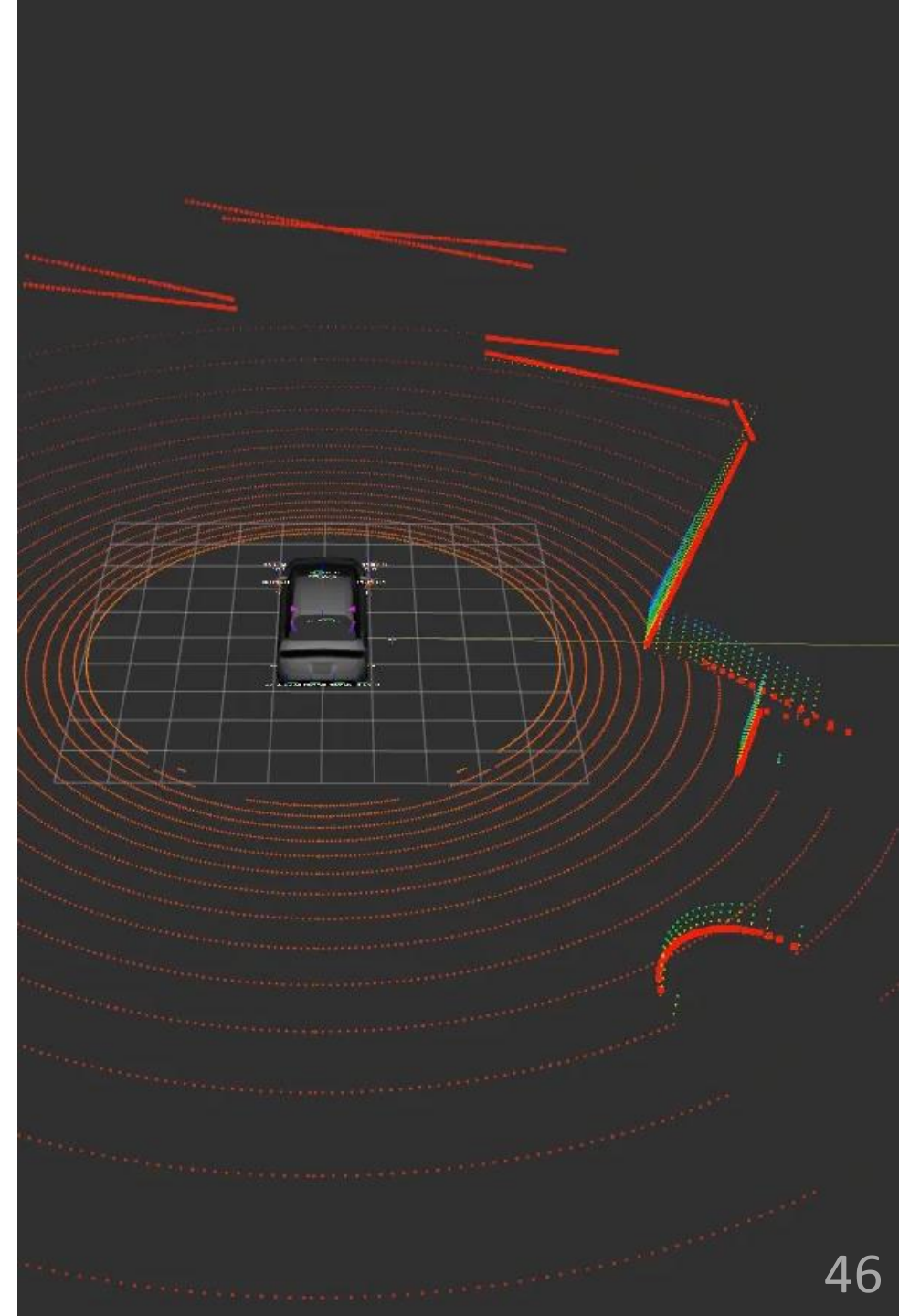
What about obstacles ? 🚧

Probability approach handles them seamlessly :



Plan

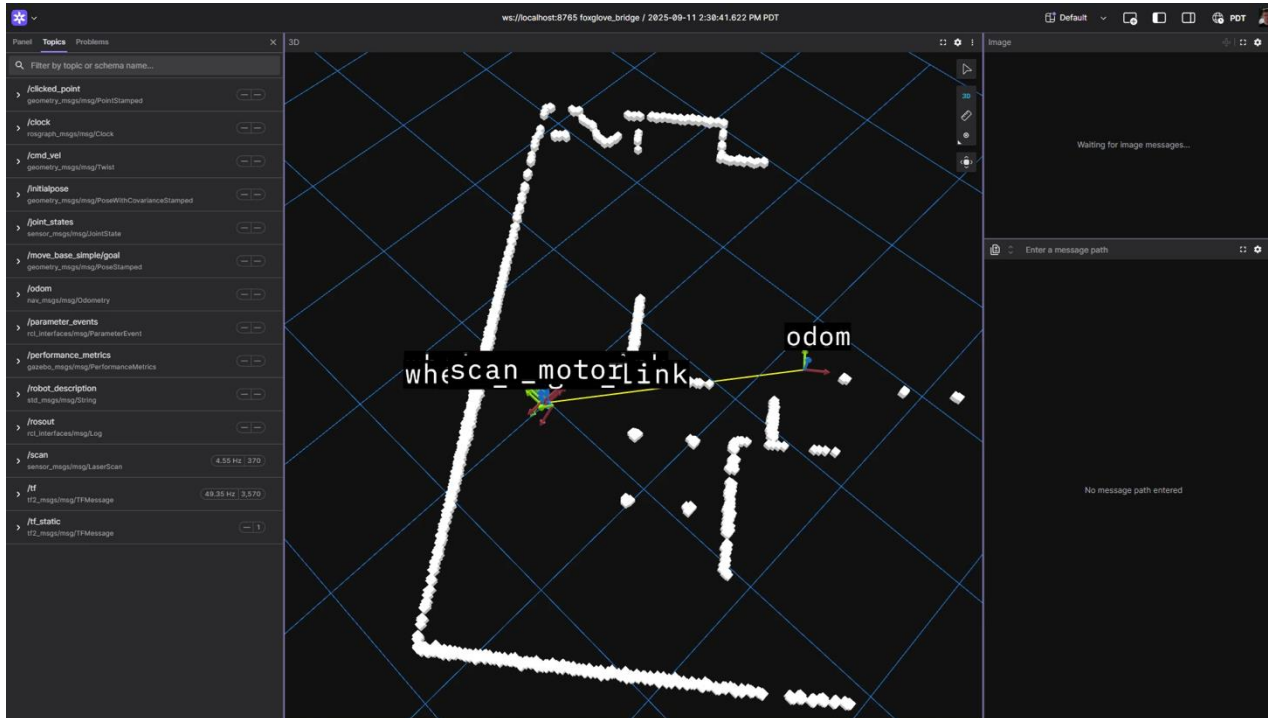
1. What is SLAM
2. The localization problem
3. The mapping problem
4. Doing altogether
5. Planning while mapping
- 6. Nav2 ROS2 library**



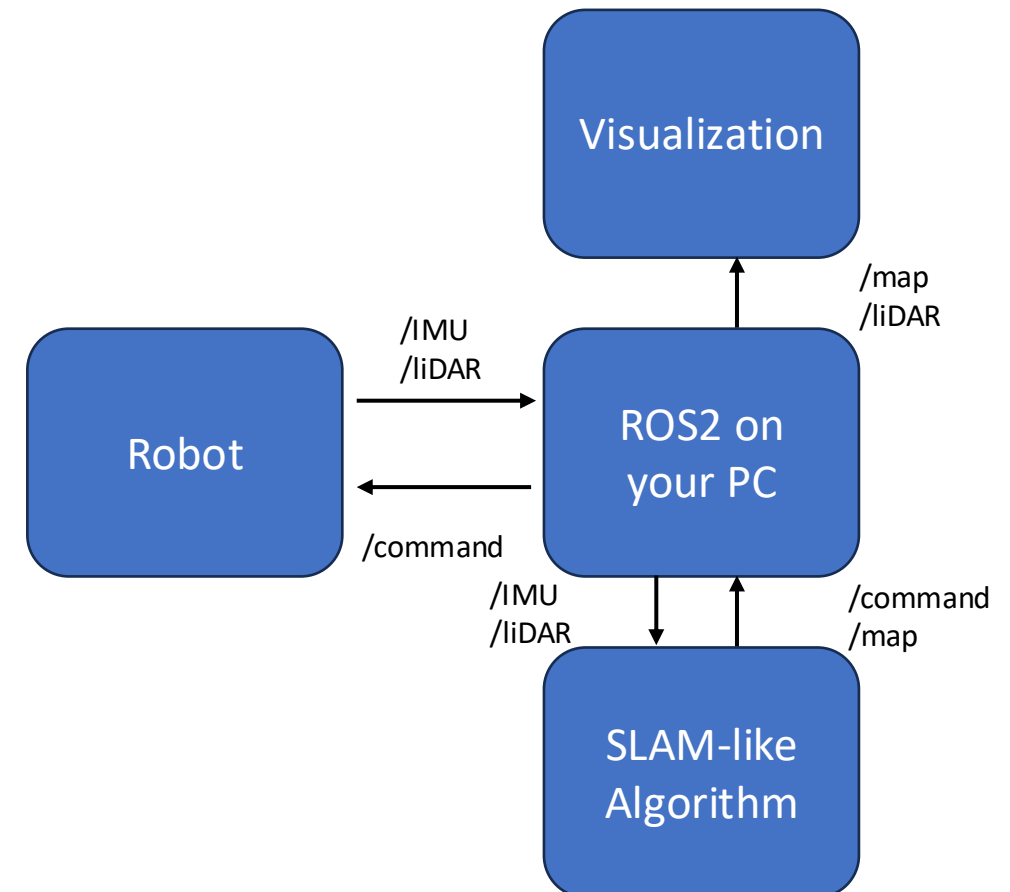
(6/6) Nav2 ROS2 library

Until now, you did :

- Command with the Remote Control
- Live LiDAR plots on Rviz/FoxGlove



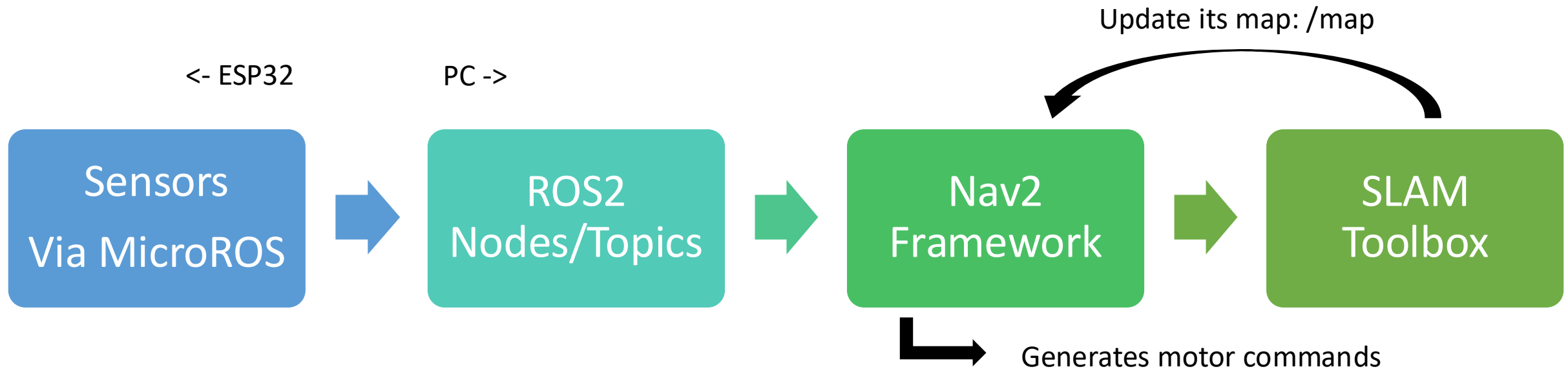
You now need to add the **logic** to **take action** from your sensor data !



(6/6) Nav2 ROS2 library

We propose the following approach to help you:

Use the ROS2 Nav2 library + ROS2 SLAM Toolbox



**Caution, NAV2 requires : ROS2
Humble/Kilted Kaiju/Jazzy Jalisco
See [here](#)**

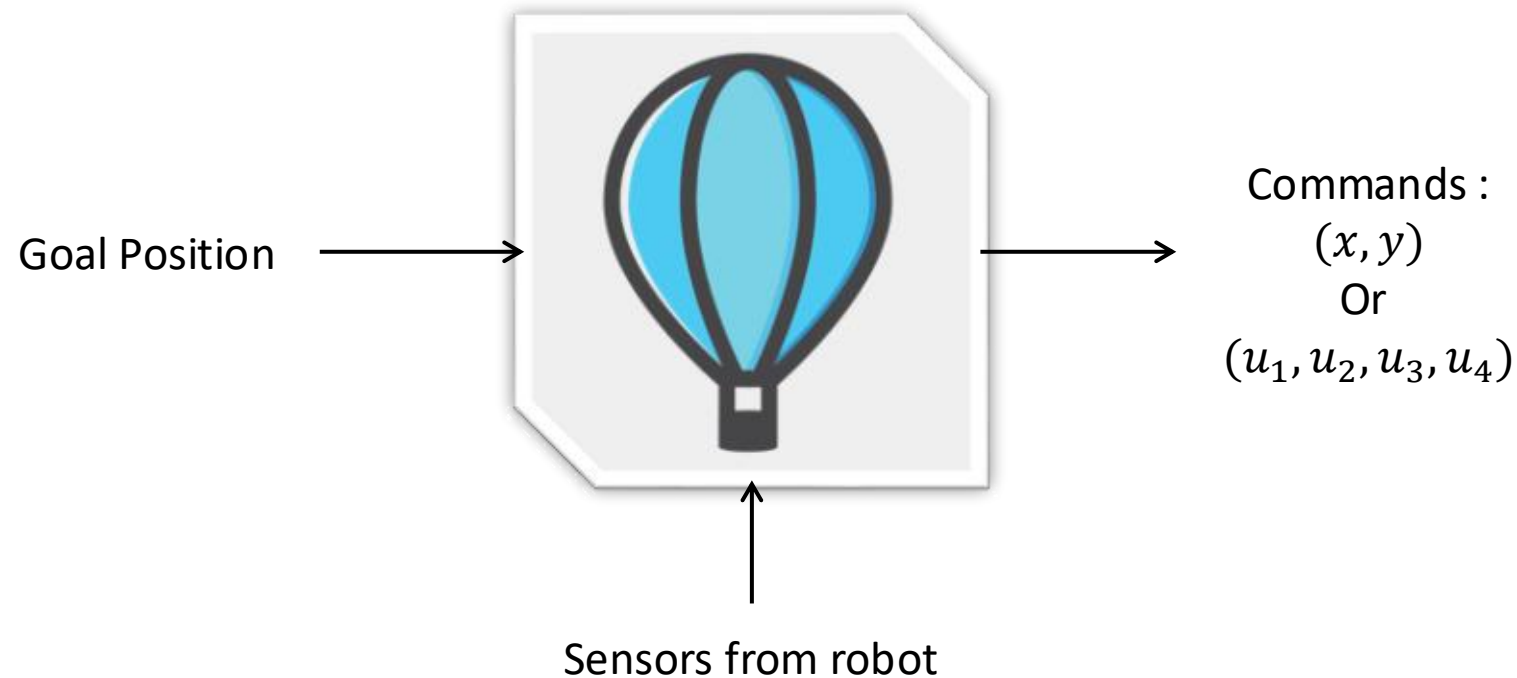
(6/6) Nav2 ROS2 library

What is Nav2 doing ?

Well...

All bricks you might need :

- **Perception**
- **Planning**
- **Control**
- **Localization**
- **Visualization**
- ...

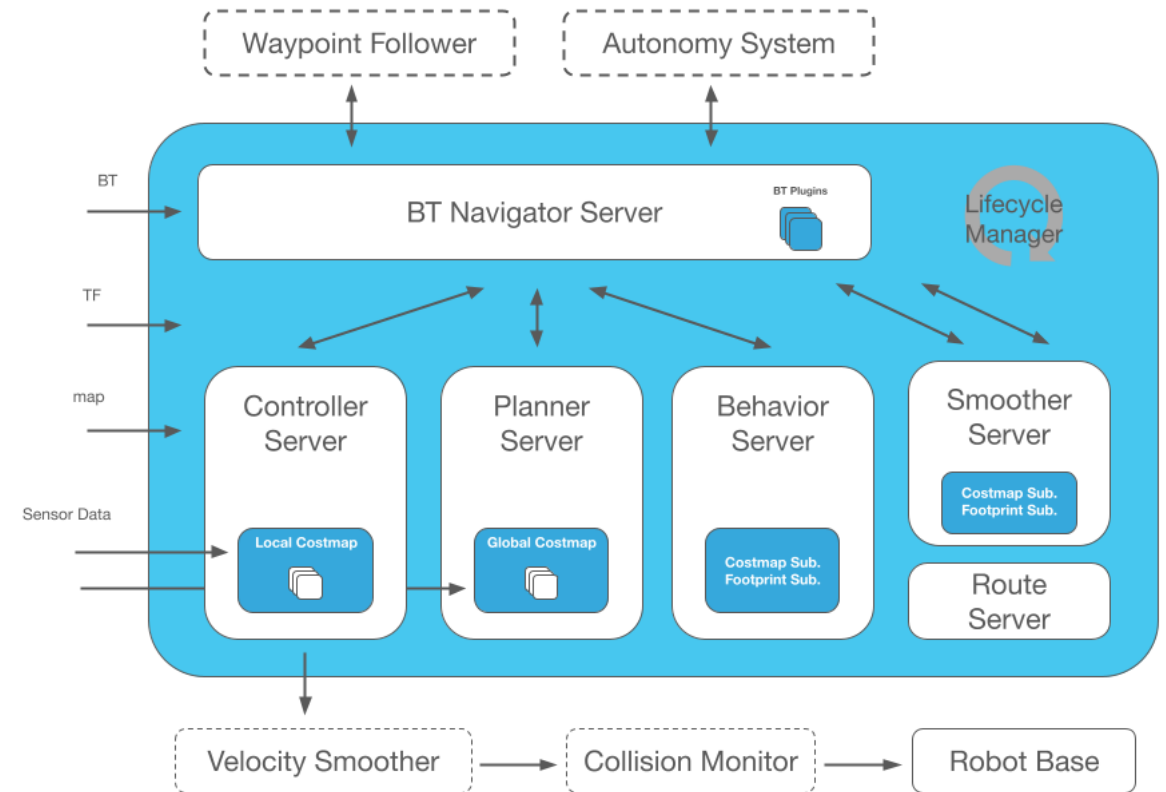


(6/6) Nav2 ROS2 library

Nav 2 Organization :

Server	Role	Mandatory
Planner Server	Computes path	Yes
Controller Server	Generates velocity	Yes
Smoother Server	Improves path shape	No
Behavior Server	Handles failures	No
BT Navigator	Orchestrates behavior	Yes

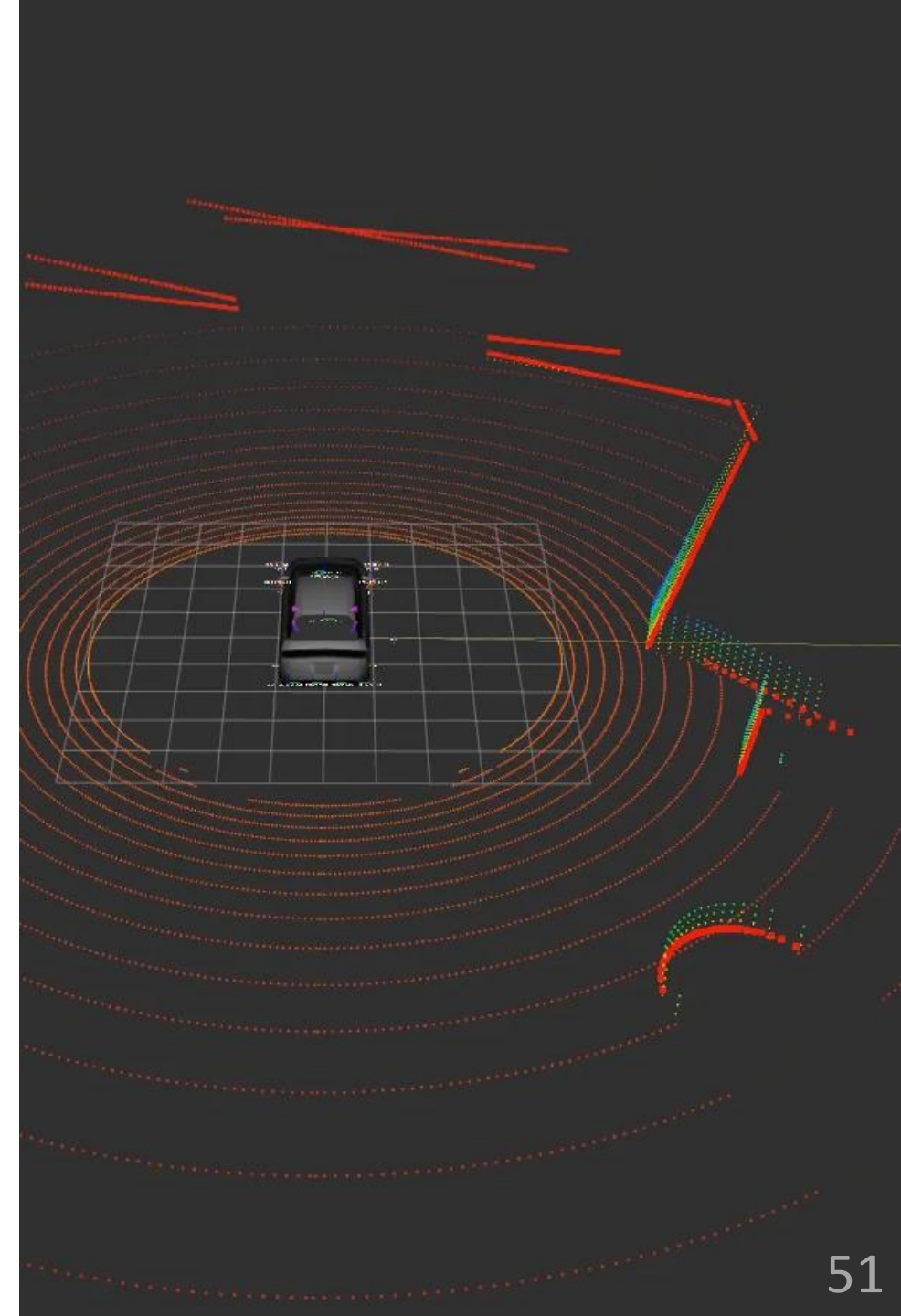
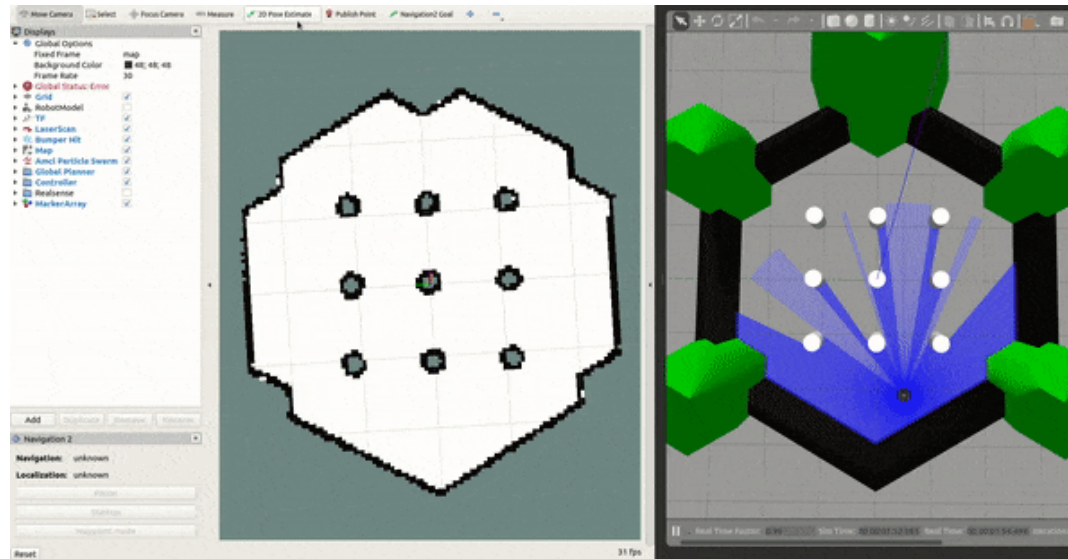
Nav 2 provides you all the bricks, on which you can plug clever algorithms



(6/6) Nav2 ROS2 library

Nav 2 starter :
[Documentation](#)
[Installation](#)

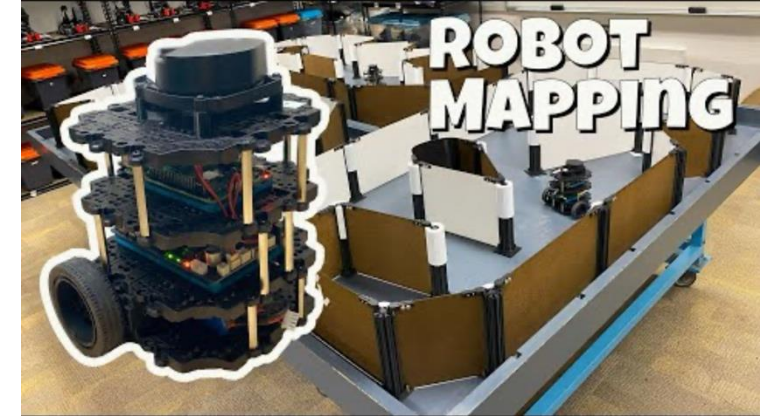
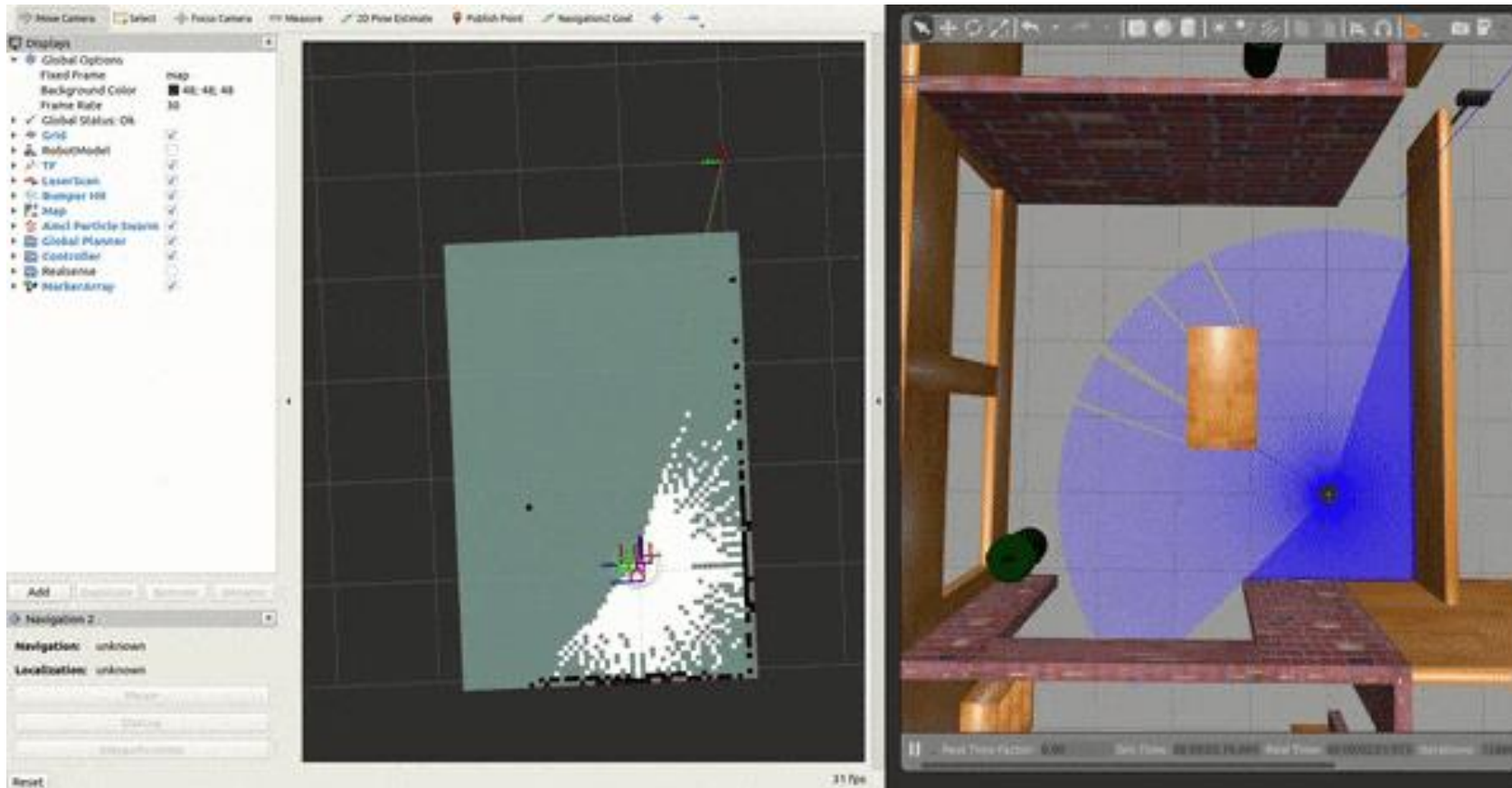
- Getting Started
- Development Guides
- Navigation Concepts
 - ROS 2
 - Behavior Trees
 - Navigation Servers
 - State Estimation
 - Environmental Representation
 - Nav2 Academic Overview
 - First-Time Robot Setup Guide



(6/6) Nav2 ROS2 library

ROS2 SLAM toolbox can be used on top of Nav 2 :

[Tutorial on Slam Toolbox](#)
[Project](#)



Quick recap

1. What is SLAM
 - Localization and Mapping at the same time**
2. The localization problem
 - Use Sensor fusion ! Often Kalman Filters**
3. The mapping problem
 - Construct grid cell maps from LiDAR data, algorithms handle them with probability**
4. Doing altogether
 - Perform naive search to construct the map and locate the robot**
5. Planning while mapping
 - You can compute a trajectory even on an incomplete map !**
6. Nav2 ROS2 library
 - Be lazy, use built-in libraries 😊**



Next steps for you project:

1. Ask questions about today's lecture if you have 😊
2. Install Nav2:
 - I. Check the NAV 2 installation guide and documentation :
 - <https://docs.nav2.org/>
 - https://docs.nav2.org/getting_started/index.html#installation
 - II. Complete with tutorials and videos if needed:
 - <https://www.youtube.com/watch?v=jkoGkAd0GYk>
 - <https://roboticsbackend.com/ros2-nav2-tutorial/>
4. Install SLAM Toolbox
 - [Tutorial on Slam Toolbox](#)
 - [Project](#)



List of sources :

Nav2 :

<https://docs.nav2.org/>

<https://www.youtube.com/watch?v=jkoGkAd0GYk>

<https://roboticsbackend.com/ros2-nav2-tutorial/>

Installing Nav2 :

https://docs.nav2.org/getting_started/index.html#installation

Installing SLAM Toolbox:

https://docs.nav2.org/tutorials/docs/navigation2_with_slam.html

SLAM in ROS2/NAV2/SLAM Toolbox in python:

<https://www.youtube.com/watch?v=xqjVTE7QvOg>

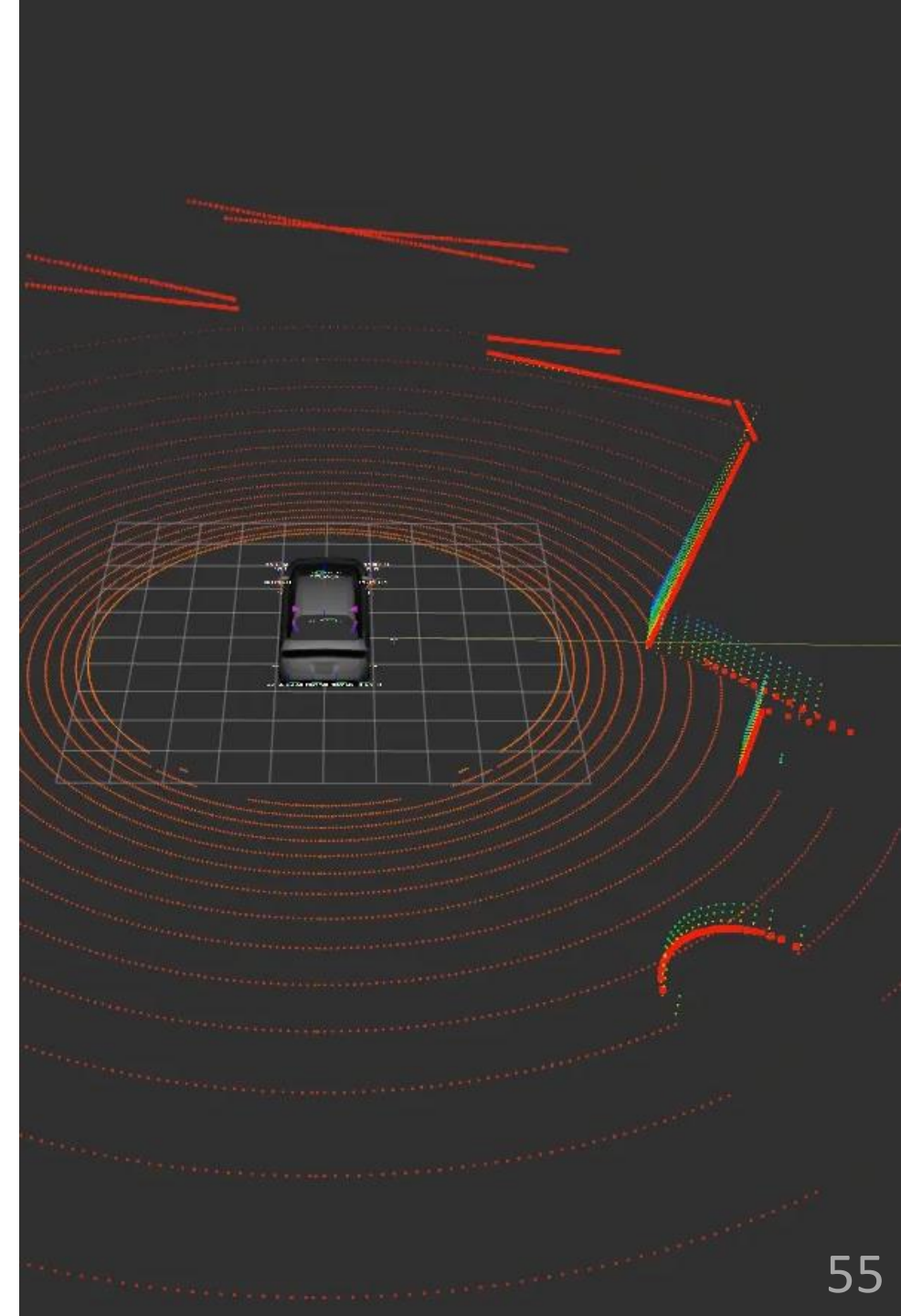
https://github.com/KaiNakamura/slam_robot

Baye's rule:

<https://www.youtube.com/watch?v=HZGCoVF3YvM>

List of some SLAM algorithms:

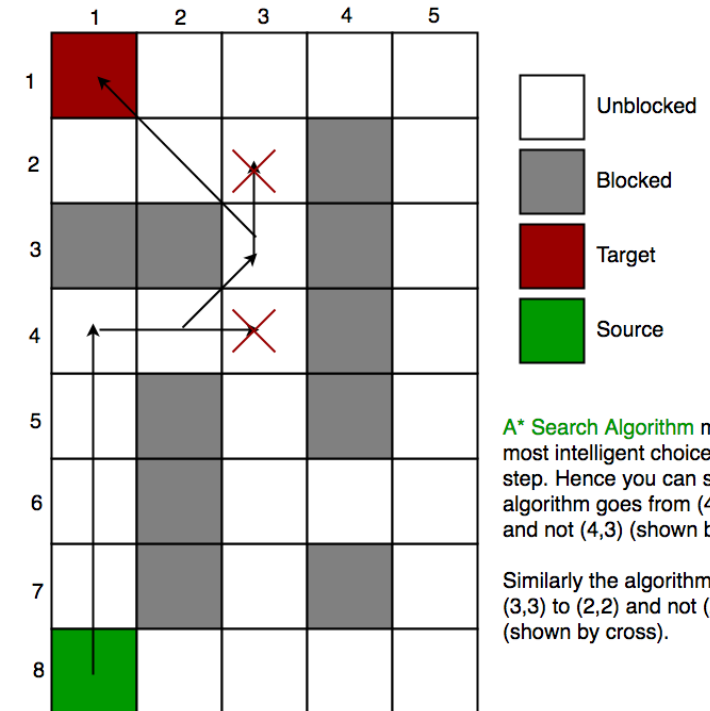
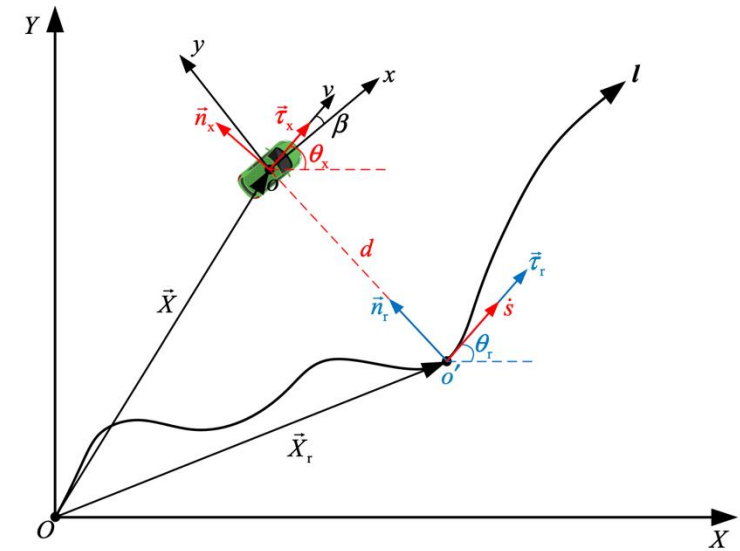
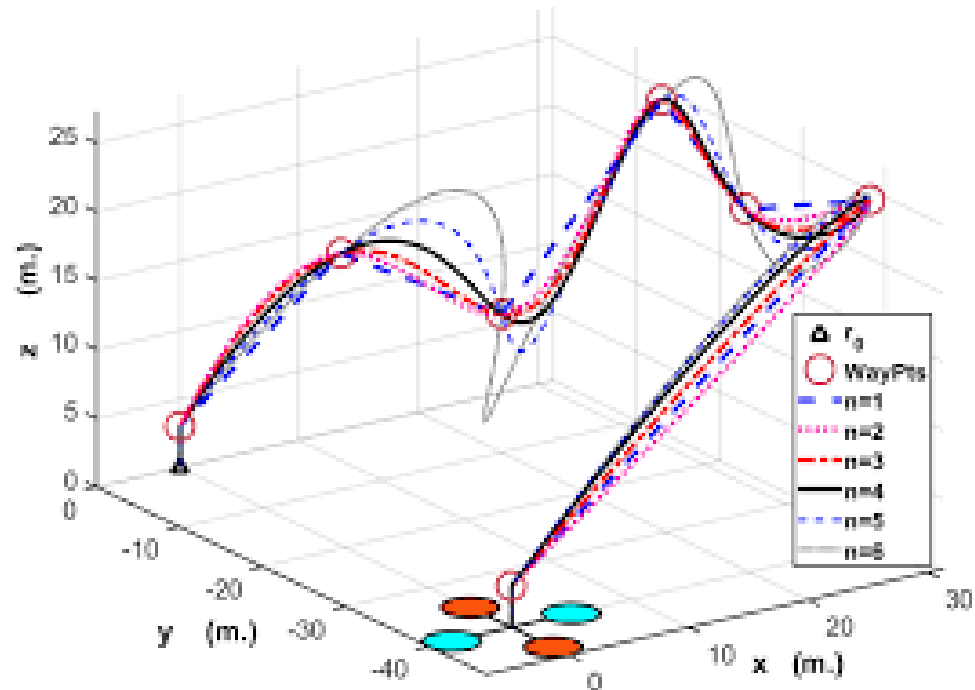
<https://openslam-org.github.io>



What's next

You will also have a short lecture on trajectory, including :

- Trajectory computation
- Shortest Path algorithms
- Trajectory tracking
- Live recomputation



A* Search Algorithm makes the most intelligent choice at each step. Hence you can see that algorithm goes from (4,2) to (3,3) and not (4,3) (shown by cross).

Similarly the algorithm goes from (3,3) to (2,2) and not (2,3) (shown by cross).

The background of the slide is a dark blue gradient with a complex, light blue technical drawing overlay. This drawing includes various geometric shapes such as circles, arcs, and polygons, along with lines that resemble a circuit board or a mechanical blueprint. The overall aesthetic is technical and engineering-oriented.

Questions?
etchassaing@omnesintervenant.com